Module Code & Module Title

CS5002NT – Software Engineering

Assessment Weightage & Type

Individual Coursework (35%)

Year and Semester

2025-26 Autumn

30 - Credit

Student Name: Manoj Katuwal

London Met ID: 24045922

College ID: np05cp4a240306

Assignment Submission Date: Apr 30, 2026

Assignment Due Date: Apr 30, 2026

I confirm that I understand my coursework needs to be submitted online via my second Teacher under the relevant module page before the deadline for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded

Table of Contents

Table of figures

Table of tables

1 Introduction

Technology is being increasingly used by businesses to improve their daily operations, as well as to deliver efficient services to their customers. The restaurant industry has also been seen significant growth in the demand for online services like order management, table reservations because customers likely to get these benefits without show up in person or make a phone call. Likely, Gokyo Bistro is a restaurant struggling with issues related to table allocation, managing large numbers of customers and meeting the growing demand for home food delivery. The absence of an effective system leads to inconvenience to customers with problems like table unavailable and slow service. This project proposes to develop a software system to automate the restaurant's operation.

This report uses Object-Oriented Analysis and Design (OOAD) to specify the system. This report consists of several software engineering documents including Work Breakdown Structure (WBS), Gantt Chart, Use Case Diagrams, Sequence Diagrams, Activity Diagrams and Class Diagrams. In addition, it proposes an appropriate development methodology, system architecture, design patterns, testing and maintenance plan to ensure the successful deployment of the system.

2 Aim and Objective

Aim:

The main aim of this report is to create a full Object-Oriented Analysis and Design (OOAD) report of the Gokyo Bistro Management System, which will be used as a guide to creating an online system to facilitate the running of restaurants including table reservation, order management, home delivery, payment processing, financial reporting and customer feedback.

Business Objective for Gokyo Bistro:

- To minimize the waiting time of customers by providing an opportunity to book a table online.
- To grow revenue through home delivery and prepayment.
- To enhance retention of customers by providing benefits of membership, beverage selections, and rating facilities.
- To improve decision making by giving the financial reports and estimation of revenue to the administration.
- To automate functions through the incorporation of menu control, order processing and billing.

3 Work Breakdown structure (WBS)

A Work Breakdown Structure (WBS) is a project management tool that breaks a large and complex project into smaller, manageable task. Instead of considering a project as a single unit of task, the WBS breaks it down into phases, and each phase is further split into specific tasks. (geeksforgeeks, 2024)

The WBS is a hierarchical breakdown, as root of the structure is labeled by the project name and breaking down into deliverables. It's broken down into smaller and smaller tasks until they can be assigned, estimated and tracked.

For our project, the WBS enabled us to discover all the tasks required to create a management system for Gokyo Bistro, including requirements gathering, design, development, testing and deployment. It was also used to develop the Gantt Chart, as the WBS tasks were used to schedule the work to be done.

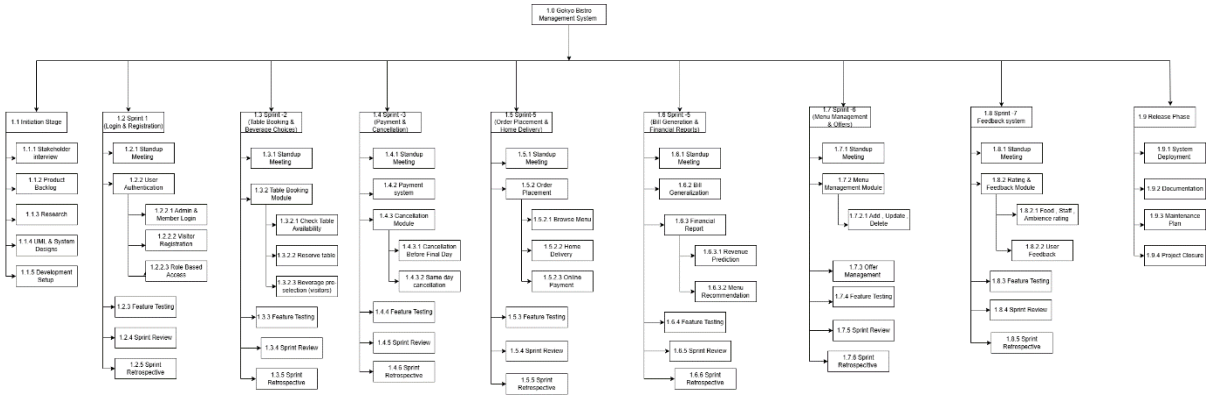


Figure 1:WBS

4 Gant Chart

A Gant Chart is a visual project management tool that represents tasks along a timeline using horizontal bars which is useful for planning and organizing all kinds of projects. It provides a graphical representation of the various task and their timing as it is simple to understand and allows easy identification of any delays in the work process. For both project managers and employees, the Gantt chart helps keep track of how the project is progressing at moment.

For the Gokyo Bistro Management System, the Gantt Chart has been prepared based on Agile Scrum Methodology. The Work is divided into sprints and each sprint focuses on delivering a specific set of features. The Chart covers the timeline from 1st February 2026 to 28th April 2026 excluding Saturdays as non-working days.

Chart:

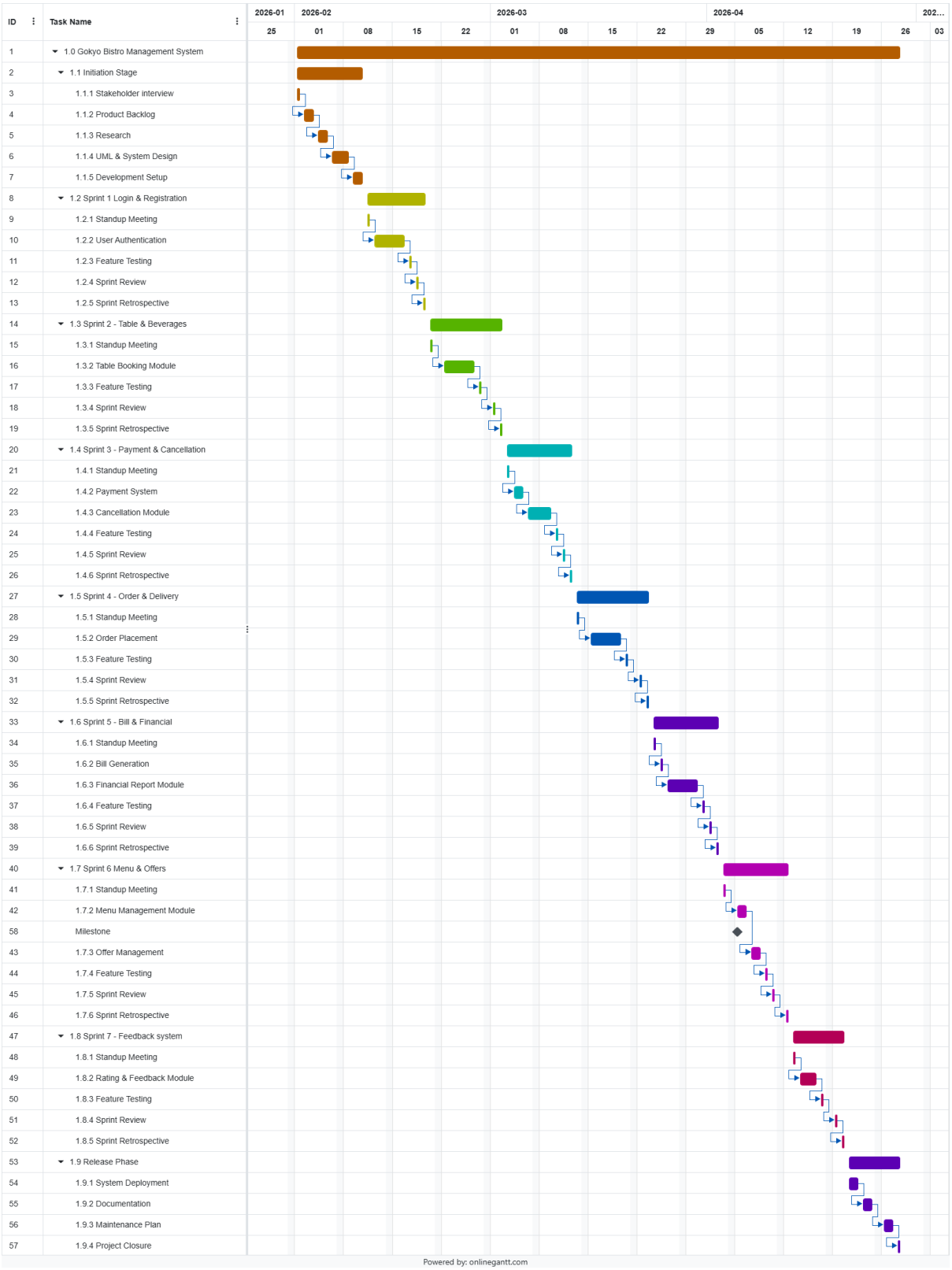


Figure 2:Gant Chart

Description of the chart:

The Gantt chart of the Gokyo Bistro Management System includes a period of about twelve weeks beginning from 1st February 2026 to 28th April 2026. All weekends have been excluded from the working days which means Sunday to Friday are counted as working days.

The projects start from the Initiation stage from 1st February to 10th February where stakeholder interviews are conducted, product backlog has been released, UML and system design has been carried out and the development plan has successfully setup.

After the initiation stage, the development phase begins with Sprint 1 targeting the Login & Registration module since it is the highest priority feature since the rest of the system's features are dependent on this particular feature. The subsequent sprints are developed on the basis of each previous sprint, adding new features at the conclusion of every two-week period.

The first milestone is achieved on April 6, 2026, when all the analysis and design documents are completed according to the coursework specifications. Sprints 5, 6, and 7 involve developing the billing & reporting module, the menu management module, and the rating & review module respectively.

The release phase begins on 21th April 2026 includes system deployment, documentation and maintenance planning. The projects end on 28th April 2026 with the final report submission due on 30th April 2026.

5 Use case diagram

A Use Case Diagram is a Unified Modeling Language (UML) diagram that shows a visual way of how users (actors)interact with the system. It illustrates the interaction between actors and the different use cases or features of the system. It is one of the earliest diagrams created in the analysis phase of a project as it defines the boundaries of the system.

In the Gokyo Bistro Management System (GBMS) Use Case Diagram, there are three actors - User, Visitor and Member. The User actor is a superclass, from which the Visitor and Member actors are derived, meaning they share some interactions with the system but also have specific interactions. The Admin is also an actor responsible for the system's back-end.

The diagram features several use cases with various relationships. Include relationships are used when one use case always includes another, like Book Table including Check Availability and Make Payment. Extend relationships are used to show optional or conditional use cases, such as Set Beverage Choices extending Book Table, or Feedback extending Rate the Experience. Inheritance relationships are used between User, Visitor and Member to indicate common interactions.

The diagram contains all the major use cases of the system such as, Registration, Login, Book Table, Place Order, Cancel Reservation, Rate the Experience, Update Menu, Generate Report, Bill Generation, and Post Offers and Schemes, which should give you a good understanding of the system and the actors who use it. (geeksOfgeeks, 2026)

Use Case Diagram Notations:

1. Actor

In use case diagram, Actor are external entities that interact with the system from outside like a person or an external system. An actor is represented by a sticky figure. In the Gokyo Bistro Management System, the actors are User, Visitor, Member and Admin.

2. Use Case

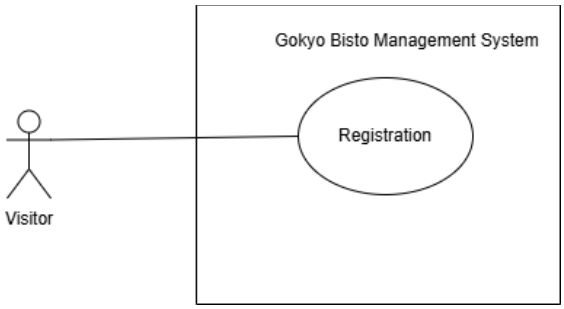
The use cases are the features or services of a system. Each use case explains a particular task that the system is able to carry out following a request made by an actor. They are used to demonstrate the system behavior as viewed by the user and they are symbolized by oval shapes such as Book table or Bill Generation.

3. Boundary

The system boundary of Use case diagram is represented by an rectangle box. It isolates the system functions to internal and external parties (actors). The name of the system should be mentioned on the top of the system.

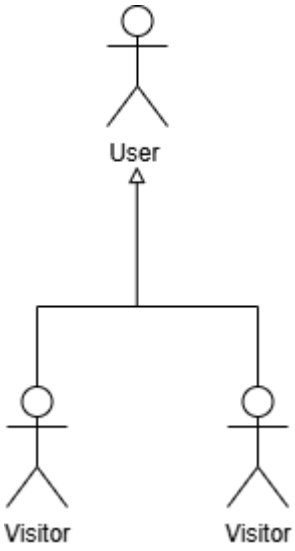
4. Association

An association in Use Case Diagram is represented by a simple line or a line with arrows at both end connecting an actor to a use case.



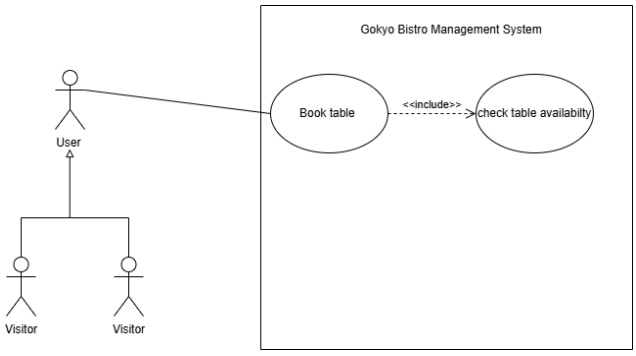
5. Generalization

Generalization in use case diagram is represented as a solid arrow with a hollow arrowhead. Generation means inheritance between actors or use cases. In the following diagram, both Visitor and Member inherit from the user actor which means they share some common features but also have their specific features or roles.



6. Include relationship

`The include relationship is shown as a dashed arrow with the label <<include>>. The include relationship adds additional functionality or use case which is not specified in base use case. The relation mean that one use case always require another use case to function. For example Book table always include Check Availability or Book table always include Date And Time availability.



7. Extend relationship

The extend relationship is shown as a dashed arrow like include relationship but with the label <<extend>>. It represents optional or conditional behavior that may be added to a use case. For example, Feedback extends Rate the experience, meaning feedback is not always required.

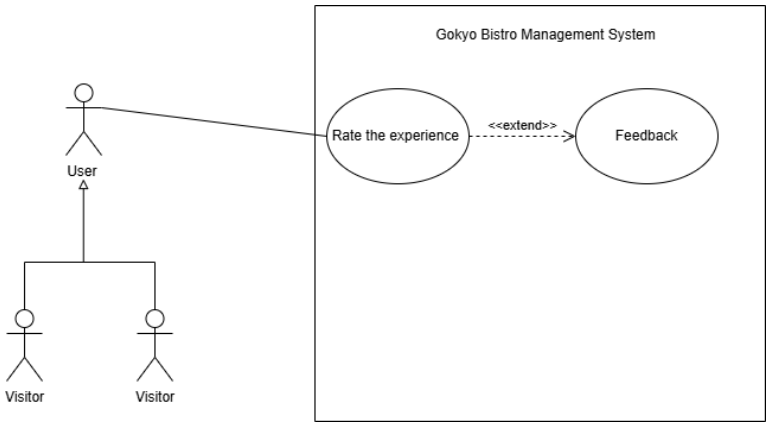


Diagram:

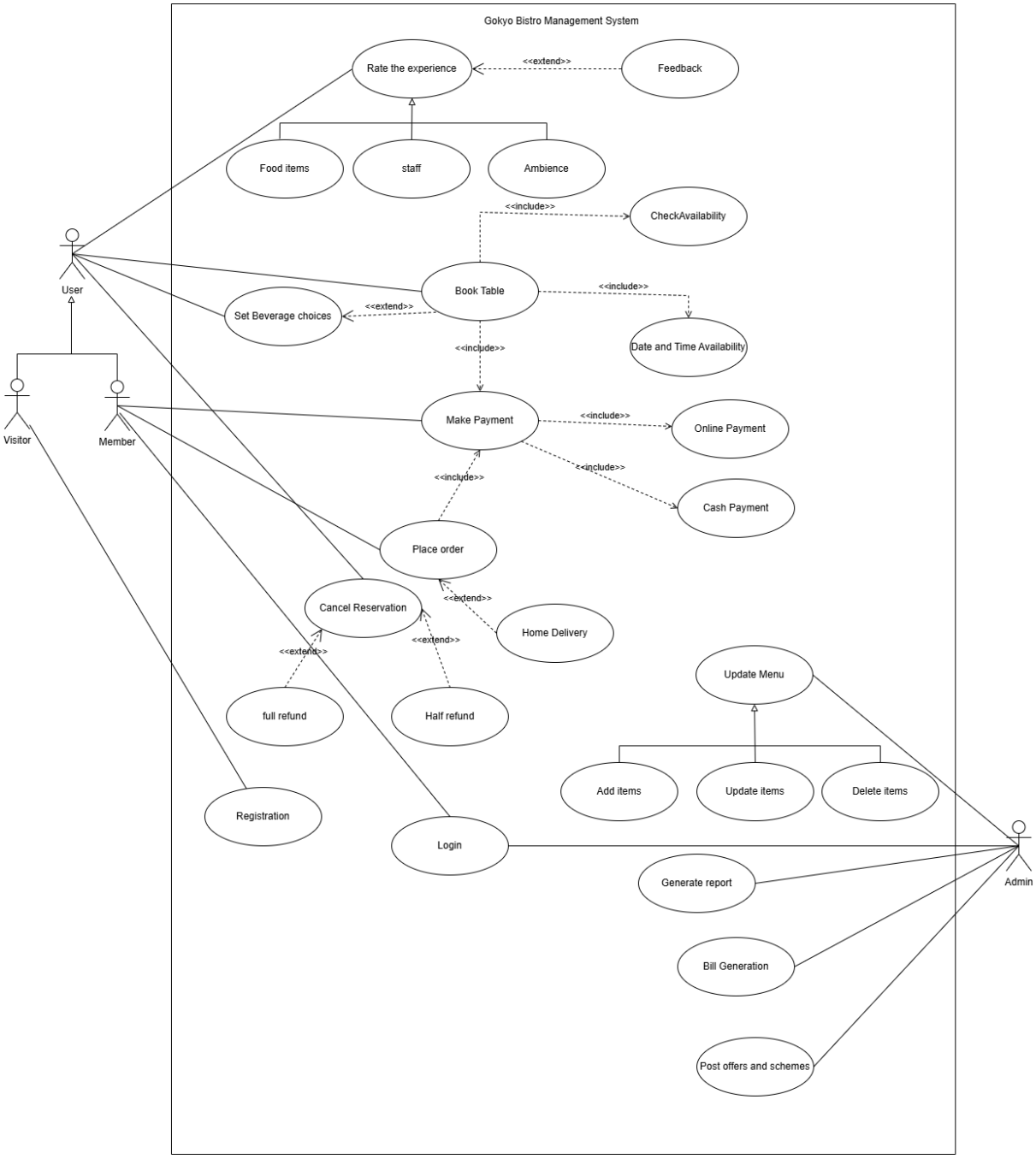


Figure 3:Use Case Diagram

5.1 High Level Use Case Diagram

High Level Use Case Description is a short summary of all the use cases identified in the Use Case Diagram. It includes the names of the actors and use case, and the system's role in the use case in brief. The table below gives the high-level descriptions of the use cases in the Gokyo Bistro Management System.

1. Registration

Use case Name	Registration
Actor	Visitor
Description	A visitor registers on the system by providing their personal details.

2. Login

Use Case Name	Login
Actor	Member, Admin
Description	Registered members and admin log into the system using their credentials to access their features.

3. Book Table

Use Case Name	Book Table
Actor	Visitor, Member
Description	Users check table availability and reserve a table for a specific date and time.

4. Set Beverages Choices

Use Case Name	Set Beverages Choices
Actor	Visitor, Member
Description	Users can select their preferred beverages from the availability list while booking a table.

5. Make Payment

Use Case Name	Make Payment
Actor	Visitor, Member
Description	Users can pay an advance of Rs 1200 to confirm their table reservation.

6. Cancel Reservation

Use Case Name	Cancel Reservation
Actor	Visitor, Member
Description	User can cancel their reservation and if cancellation is done before the final day full refund is given but if cancelled on the same day then then Rs 500 is deducted and the remaining amount will be given.

7. Place Order

Use Case Name	Place Order
Actor	Member
Description	Members can place food and beverages orders where home delivery is optional.

8. Home Delivery

Use Case Name	Home Delivery
Actor	Member
Description	Members can request their order to the delivered to their home address.

9. Rate the Experience

Use Case Name	Rate the Experience
Actor	Visitor, Member
Description	Users can rate the overall experience including food items, staff behavior, and ambience and also submits text feedback.

10. Manage Menu

Use Case Name	Manage Menu
Actor	Admin
Description	Admin can manage the menu by adding items, deleting items or updating existing items.

11. Generate Report

Use Case Name	Generate Report
Actor	Admin
Description	Admin can generate financial reports and do revenue predictions for future years with menu recommendations.

12. Bill Generation

Use Case Name	Bill Generation
Actor	Admin
Description	Admin can generate bills for dine-in customers and receipts for order placed through the system.

13. Post Offers and Schemas

Use Case Name	Post Offers and Schemas
Actor	Admin
Description	Admin can post special offers and discount schemas on the system for customers.

5.2 Expanded use case diagram

Expanded use case description provides a detailed step by step account of how every use case is carried out. Like high level use case description it includes short description of the use case and also includes main flow of events. Login and Cancel Reservation has been chosen for the expanded description.

1. Login

Use Case Name: Login

Actor: Member, Admin

Description: Registered members and admin log into the system using their credentials to access their features.

Typical Course of Events:

Actor Action	System Response
User navigates to the login page.	System displays the login form with username and password.
User enters the username, password and clicks the login button.	System validates the credentials in the database.
	System creates a session and redirects the user to their respective dashboard.
User access their dashboard and uses available features.	

Alternative courses:

Line-3: If invalid credentials is provided then the system displays error message. User is prompted to try again.

2. Cancel Reservation

Use Case Name: Cancel Reservation

Actor: Visitor, Member

Description: User can cancel their reservation and if cancellation is done before the final day full refund is given but if cancelled on the same day then the Rs 500 is deducted and the remaining amount will be given.

Typical Course of Events:

Actor Action	System Response
1. Users logs into the system.	
2. User navigates to Reservation.	3. System displays the list of reservations.
4. User selects a reservation to cancel.	5. System ask for confirmation.
6. User confirms confirmation.	7. System checks cancellation date.
	8. System process full refund if cancellation is done before final day.
	9. System updates reservation status to cancelled.
	10. System notifies user to successful cancellation.

Alternative courses:

Line-3: f no reservation exists system shows ‘No active reservations’ and use case ends.

Line-5: If user click ‘No’ then use case ends.

6 Sequence Diagram

A Sequence Diagram is an important part of a Unified Modelling Language (UML) diagrams that shows the sequence and message flow between objects or components in a system. It showcases the order in which messages or events are exchanged by objects to complete a certain use case or workflow. Time proceeds from the top down on the diagram and each object has a vertical lifeline.

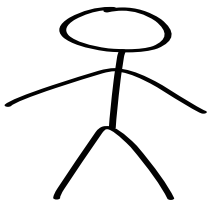
Sequence diagram are useful in software development for number of reasons as they depict how objects or system interact with each other in a sequential order. Because of their visual nature, they provide an easy way to convey system behavior which helps team members understand complex interactions without go through the actual code. It is also helpful during use analysis and representing use cases, as they clearly specific processes are executed within the system. (geeksOfgeeks, 2026)

For this project, the sequence diagram has been produced for the Cancel Reservation use case.

Sequence Diagram Notations used in Cancel Reservation Use Case:

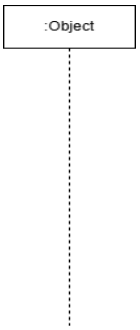
1. Actor

An actor in sequence diagram represents role where it interacts with the system and its objects. It is represented by Sticky figure.



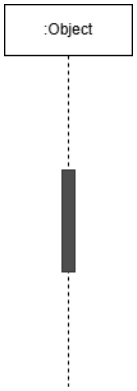
2. Lifeline

A lifeline in a Sequence Diagram models a particular participant in the interaction. It is represented by a vertical dashed line extending down from a box at the top, and it represents the object's presence during the sequence. Lifelines are used to represent and identify the objects that participate in the interaction, and the messages are represented as horizontal arrows from one lifeline to another.



3. Activation Bar

An activation bar, is a narrow rectangle found on a lifeline of a Sequence Diagram. It signifies the time when an object is working on something or waiting for a message. It begins when the object receives a message and starts working on it and is terminated when the object has completed the task and returns the control to the caller.



4. Messages

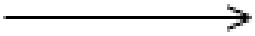
The communication between objects is depicted using messages. The messages appear in a sequential order on the lifeline. We represent message using arrows. It can flows in any direction. It has some types and they are given as below:

a. Synchronous message

A synchronous message is a type of message where the sender sends a request and waits for the receiver to process the message. It always receives a reply message. It is represented by solid arrowhead pointing from the sender lifeline to the receiver lifeline.

b. Asynchronous message

An asynchronous message is a type of message where the sender sends a request and does not wait for a response. The interaction moves forward irrespective of the receiver processing the previous message or not. It is represented by an open or half arrowhead pointing from sender lifeline to the receiver lifeline.

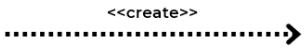


c. Return message

A return message or also known as reply message in sequence diagram represents the response sent back from the receiver to the sender. We represent return message by a dashed arrow pointing back from the receiver lifeline to the sender lifeline.

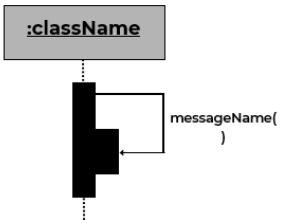
d. Create Message

A create message in a Sequence diagram is a message that is used to create a new instance of a new object during interaction.



e. Reflexive message

A reflexive message in a Sequence diagram is a message where objects send a message to itself.



5. Fragments

Fragments in a Sequence Diagram is used to model logic such as conditions or loops in the interaction. They are represented as a box with a small pentagon in the upper left corner which indicates the type of fragment. The use of fragments makes sequence diagrams more expressive by being able to express complex logic such as conditions, loops and alternatives in a sequence diagram without having to create multiple diagrams.

There are a number of different fragment types in UML but the ones used in this sequence diagram are the alt fragment and the loop fragment.

The alt fragment, which stands for alternative, is an interaction that represents a conditional statement (similar to an if-else statement in code). This is a box that has sections separated by a dashed line, each section representing a different possible flow based on a condition. Only one section will be executed depending on whether or not the condition is true. The alt fragment is used in the Cancel Reservation use case to show the results of cancelling the reservation. If the reservation is cancelled before the last day, the user is refunded the advance payment. But if the cancellation occurs on the same day, there is a charge of Rs. 500 is charged and the user is refunded the balance.

The loop fragment is used to model a loop, like in programming. This means a set of messages will be repeated until a certain condition is no longer true. In the Cancel Reservation use case, the loop fragment represents the situation where the system keeps on checking and updating the reservation status until the cancellation procedure is finalized and confirmed.

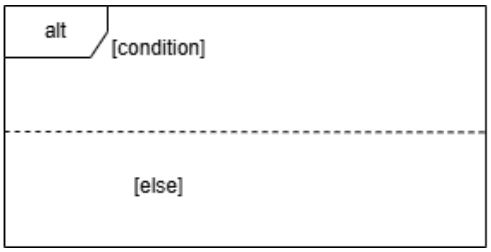


Diagram:

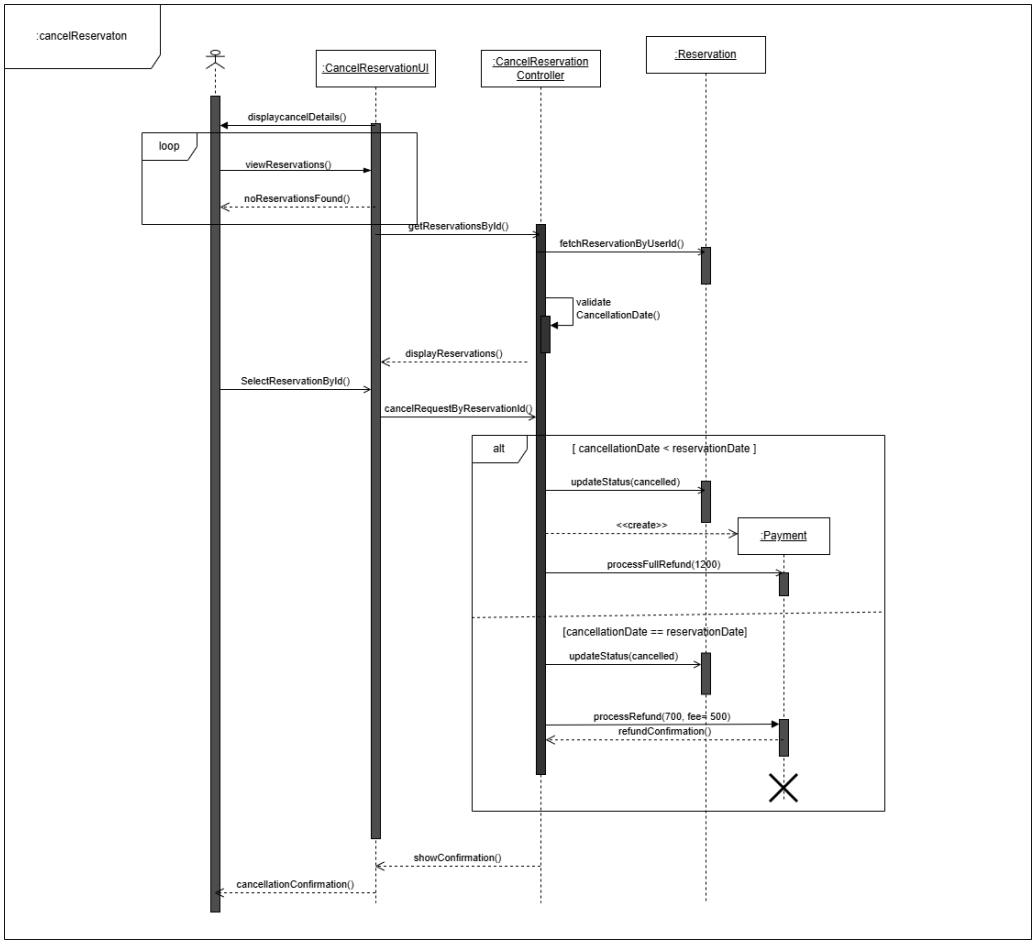


Figure 4:Sequence Diagram of Cancel Reservation

7 Activity diagram

Activity Diagram is an essential part of the UML diagram that help to illustrates the process workflows or activity flow within a system. It depicts the sequence of steps, choices and parallel actions performed by a system to complete a certain process where different actions are connected and how a system moves from one state to another. Activity diagrams can be quite helpful when it comes to designing use cases where there are various alternative flows and decisions involved as they illustrate process flow from beginning till the end in a detailed manner. In this report, an activity diagram has been generated for Rate the Experience use case (geeksforgeek, 2026).

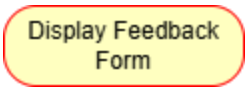
7.1 Notations used

1. Initial Node

The initial mode is the starting point of the activity diagram. It is represented by a solid filled black circle. A process can have only one initial state.

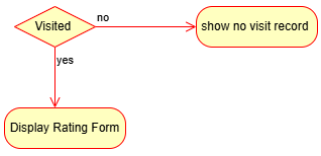
2. Activity

An activity represents execution of an action performed within the process. It is represented using a rectangle with rounded corners. Basically, any action takes place is activity. Example in this diagram include submit Rating, Enter Feedback, Display Feedback form etc.



3. Decision node

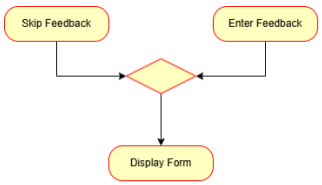
The diamond shaped symbol denotes the decision node. When we need to make a decision whether to check the flow of control we use decision node. It indicates a point where condition is checked and the flow the takes place one of two alternative paths. In this diagram three decision nodes have used, one to check whether the user has visited the restaurant, one to validate the ratings and one to check whether the user wants to provide written feedback.



Decision Node

4. Merge Node

A merge node is symbolized by a diamond but does not contain a condition. It differs from the decision node as it merges two or more activities into one if the control proceeds onto the next activity irrespective of the path chosen. In this case the node merges the Skip Feedback and Enter Feedback paths into one the Display Feedback Form activity.



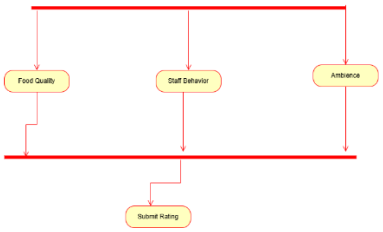
5. Fork Node

A fork node is used to support concurrent activities. It is represented by a thick horizontal bar. It divides a single flow into several parallel flows that run concurrently. In this case, the fork node divides the task of rating into three parallel tasks and they are Food Quality, Staff Behavior and Ambience which means all three can be rated at simultaneously.

6. Join Node

A join node is represented by a thick horizontal line. It supports concurrent activities converging into one. For join notations we have two or more incoming edges and one outgoing

edge. Here, the join node combines the three rating process into one and then moves on to the Submit Rating process.



J

7. Action flow

Action flow is represented by arrows connecting on activity or node to another. In this diagram, it is used to show the transition from one activity state to another activity state.

8. Final Node

The state which the system reaches a particular process or activity ends is knows as aa Final node. It is represented by a larger circle with a sold filled circle inside it which indicates that activities have been completed and process has ended.

9. Swimlanes

Swimlanes is used for grouping related activities into one column or one row. In this diagram, we split it into two vertical columns User and System to show who is responsible for each activity. This clearly maps out who is responsible for every step.

User (Visitor/ Member)	System

7.2 Activity diagram of Rate the Experience

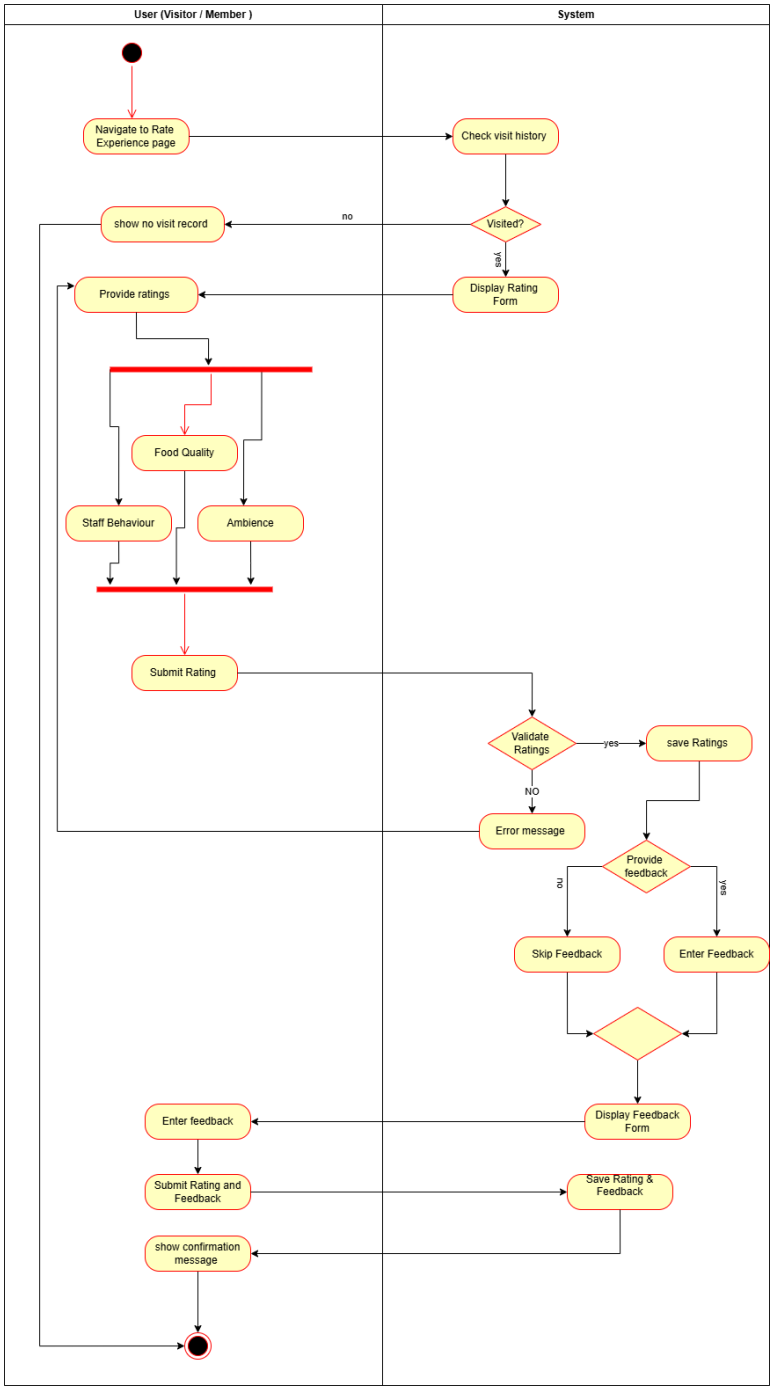


Figure 5:Activity diagram of Rate the Experience

The process starts when the user navigates to the Rate the Experience page. The system then checks if the customer has visited the restaurant before. If the answer is no, the flow will loop back and the user cannot continue. If yes then the rating page will be displayed for the customer to start the rating process. At this point, a fork node splits the flow into three parallel activity

that allow user to rate the Food Quality, Staff Behavior and Ambience at the same time. When the customer completes all three processes, the flows merged at the join node into one and the user submits the rating.

The system then validates the submit ratings, if the ratings are invalid an error message will be shown and the user is brought back to the Provide Ratings phase to try again. However, if the ratings provided are valid, then the ratings are saved in system and a decision node is made on whether or not the user wants to input written feedback. If the user chooses to provide feedback, then the user can enter the feedback whether if not then the user can skip feedback route. Then the user can submit both the rating and feedback on same time. The system will save the rating and feedback provided by the user and show him a confirmation popup message that everything went through successfully.

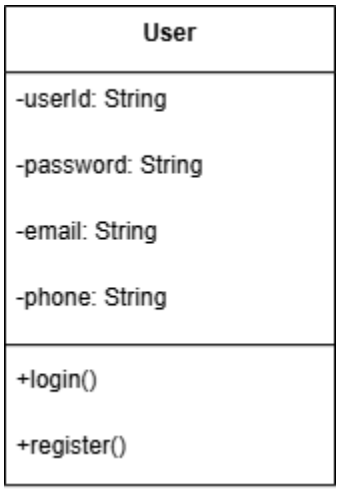
8 Class Diagram

A Class Diagram is a class of UML structural diagram, which illustrates the static structure of a system, displaying classes, attributes, methods of the system and relationships among them. It is among the most significant diagrams in Object Oriented Analysis and Design as it lays the basis on how the system shall actually be developed. The classes in the diagram are real world entities in the system, and class-relationships indicate how entities in the system interact and are dependent on one another.

8.1 Notation used

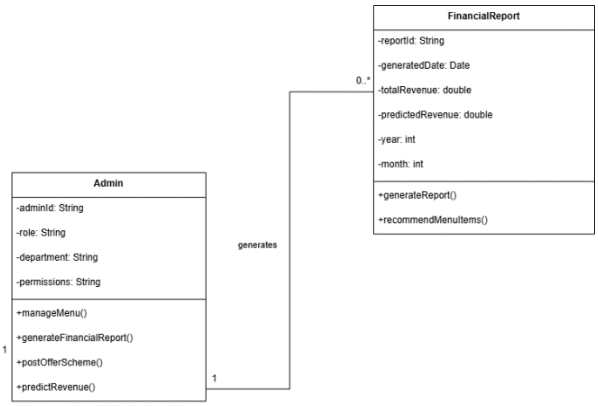
1. Class

A class is represented by a rectangle divided into three sections where the top section contains the class name, the middle section contains the attributes and the bottom section contains the methods or operations of the class.



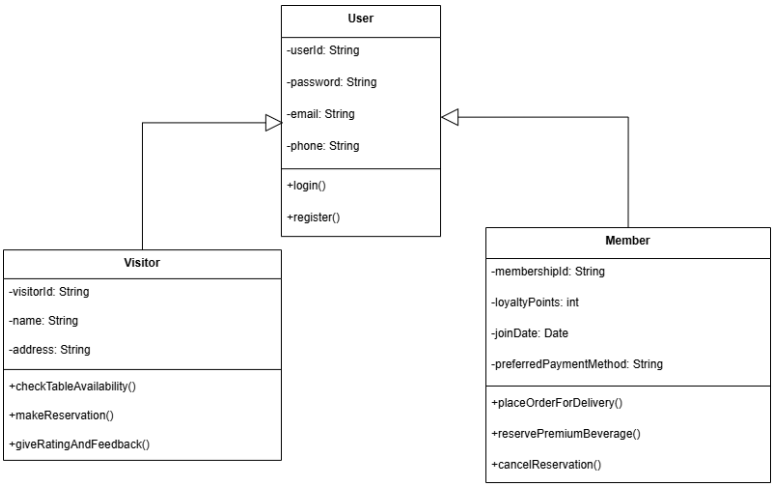
2. Association

An association is represented by a solid line connecting two classes which signifies the relationship between two classes.



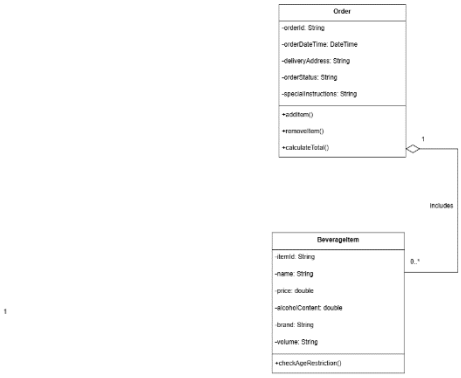
3. Generalization

Generalization is represented by a solid line with a hollow triangular arrowhead pointing toward the parent class. It shows that one class inherits the attributes and methods of another class.



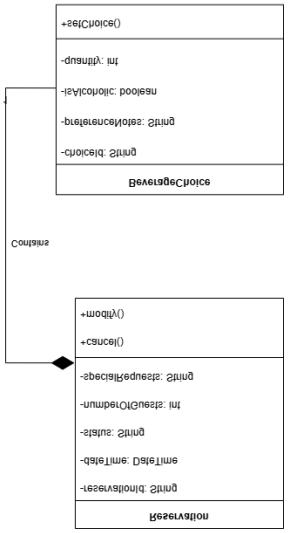
4. Aggregation

Aggregation is represented by a solid line with a hollow diamond at one end. It represents a whole-part relationship where the part can exist independently of the whole. n this relationship, the child class can exist independently of its parent class.



5. Composition

Composition is a stronger form of aggregation, indicating a more significant ownership or dependency relationship. It is represented by a solid line with a filled diamond at one side. It shows a highly interrelated relation between the whole and its parts to such an extent that parts do not have a separate existence apart from their whole.



6. Multiplicity

Multiplicity is shown at each end of an association line and indicates how many instances of one class can be associated with instances of another For example 1 means exactly one, 0..* means zero or more and 1..* means one or more.

8.2 Domain classes

Table 1: Domain class table

Class	Description
User	Base class representing any user of the system with common attributes like userId, password, email and phone.
Visitor	Inherits form user, represents an unregistered user who check availability, make reservations and give feedback.
Member	Inherits form user, represents a registered user with addition features like placing orders and reserving premium beverages.
Admin	Manages the backend operations of the system including menu management, report generation and posting offers.
Reservation	Represents a table boking made by a visitor or member with details like date, time, status etc.
Table	Represents a physical table of Gokyo Bistro with attributes like capacity, location etc.
Payment	Handles all payment transactions including advance payments, refunds and online payments.
Order	Represents a food and beverage order placed by a member with delivery and status details
FoodItem	Represents individual food items on the menu with attributes like name, price, category and availability.
BeverageChoice	Represents the beverage preferences selected by a user during table booking.
Rating	Represents the ratings and feedbacks submitted by visitors and members for food, staff and ambience.

Bill	Represents the bill generated for dine-in customer or receipt for an online order.
Financial Report	Represents the financial report generated by the admin including total revenue, predicted revenue and menu recommendations.
Offer Scheme	Represents the special offers and discount schemes posted by the admin

Diagram:

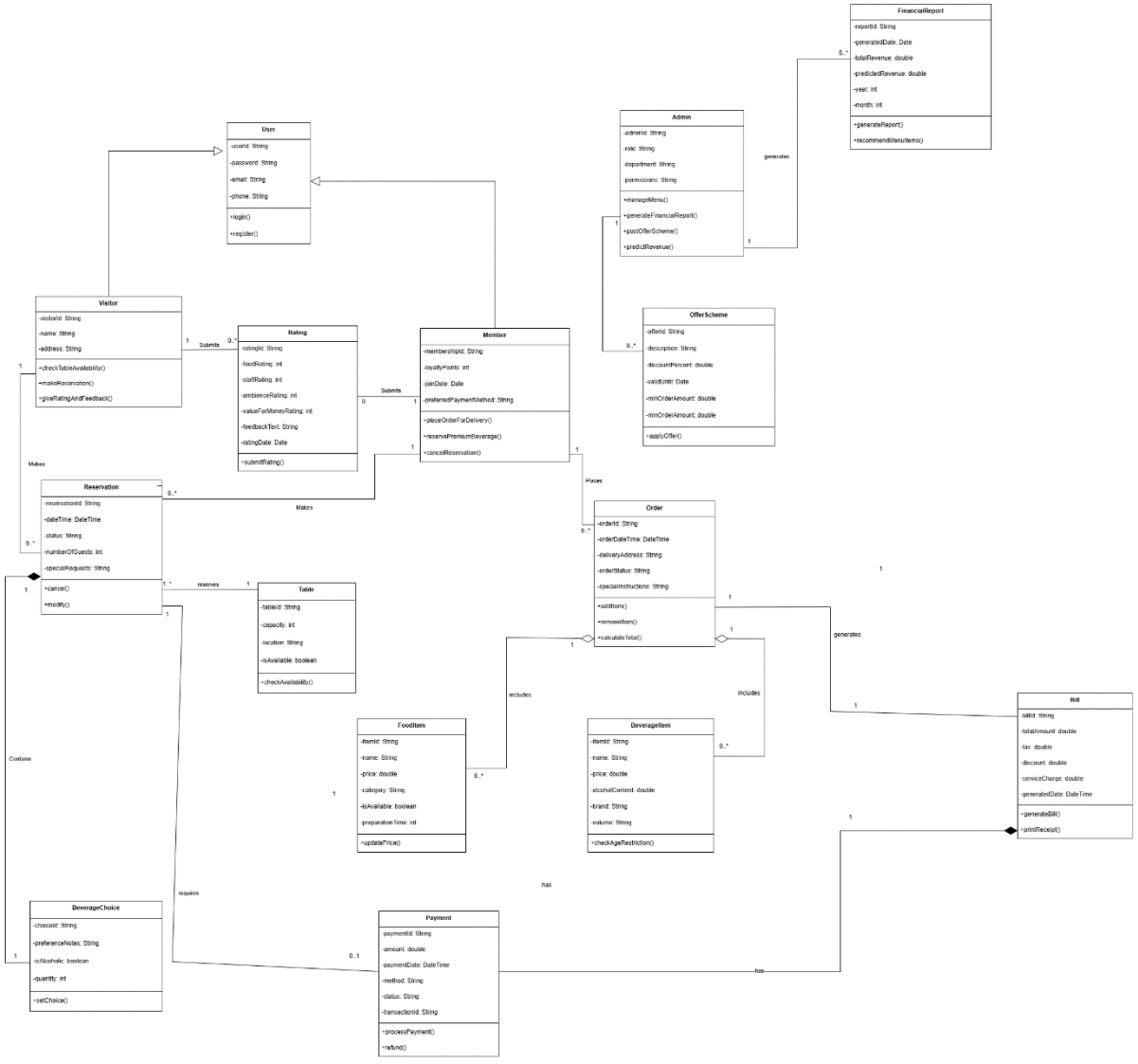


Figure 6:Class Diagram

9 Further Development Plan

9.1 Development Methodology

Agile Scrum methodology is chosen for this project. It is based on principals such as iterative method of managing projects and developing software. It enables teams to self-organize, collaborative effectively and deliver high value with helps the team to add more value to its customers through simpler processes and faster deliveries. In this method, the team does not develop the whole product in one time rather it develops the product in phases, where the customer gets to monitor the progress and give feedback.

Scrum is a widely used Agile framework for developing complex products. It facilities teamwork and self-organization of teams that can work efficiently in the process of creating high value. This approach split task into short cycles known as sprints, which are usually two weeks long, and each sprint delivers a working product with certain features. Scrum methodology has been selected for the development of the Gokyo Bistro management system since it involves several different features and the customer might want changes as he sees the system developing (geeksforgeeks, 2026).

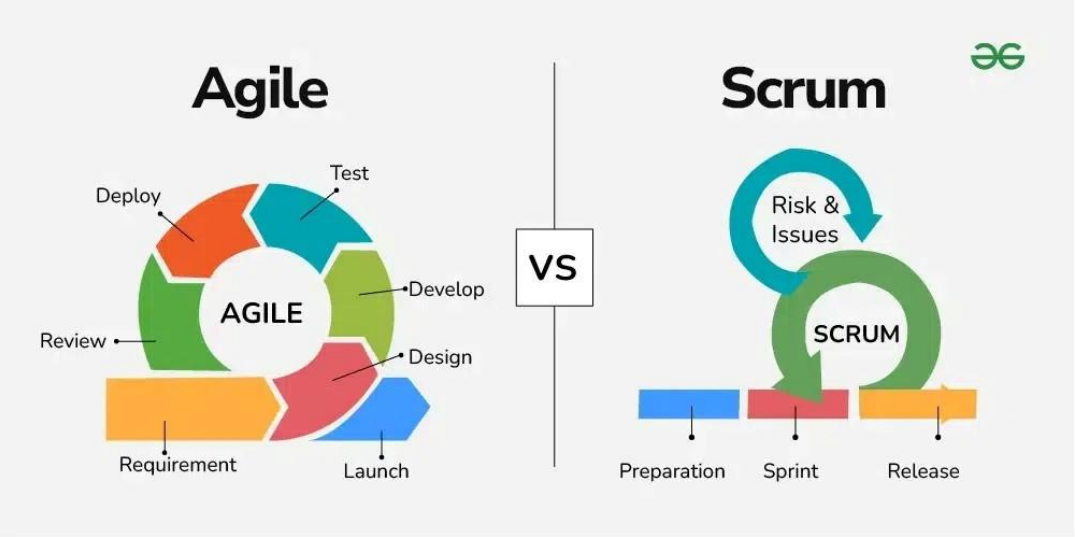


Figure 7:Agile and Scrum

9.1.1 Key Components of the Scrum

The Scrum components consist of several key components of iteration development. The following elements are used in the project:

1. Sprint

The term Sprint refers to a period of time that is used by the development team to accomplish a number of tasks chosen from the product backlog. The sprint usually lasts one to two weeks and conclude in the delivery of an increment of the software. In the Gokyo Bistro Management System, the sprints are designed in such a way that each one of them addresses certain features of the system like Login & Registration, Table Booking & Beverages Choices etc.

2. Sprint Review

Sprint Review is held after each sprint, during which the development team showcases the completed and functioning increment to the client. In our case, the functionalities will be demonstrated to Gokyo Bistro. This will ensure that the system under development conforms to the client's requirements and makes it possible for the development team to make changes to the upcoming sprint.

3. Sprint Retrospective

The Sprint Retrospective is a meeting which takes place after the Sprint Review. In this meeting, the team analyses its performance in the previous sprint and identifies the elements that went right and those which didn't. Based on this analysis, the team tries to come up with ideas for improving performance in the upcoming sprint. This Continuous improvement is a key strength of Scrum.

4. Product Backlog

Product Backlog consists of prioritized elements that need to be achieved in the development process for Gokyo Bistro Management System. It is maintained by the Product Owner and remains constantly up-to-date during the entire project. The topmost entries in the Product Backlog are of the highest priority and will be implemented first. For example, Login & Registration and Lower priority elements such as Feedback System will be handled later in other sprints.

9.2 Architectural Choice

After determining the development methodology, the next important decision involves selecting appropriate software architecture for the system. It defines fundamental organization of the system and more simply defines a structured solution as it determines how the various components of a software system are assembled. Essentially, it serves as a blueprint for application of how the system will be structured and how its components will interact with each other within the system. It is essential to select the appropriate software architecture since it determines scalability, maintainability and testability of the system (geeksofgeek, 2025). The architecture design adopted by the Gokyo Bistro Management system is the Layered Architecture Design.

9.2.1 Layered Architecture

Layered Architecture are said to be the most common and popular architectural framework in software development. It is also known as n-tier architecture and also organizes the system in several separate horizontal layers that function together as a single unit of software. Each layers plays a different role in the operation of the application. Data is flow from the upper level to lower level. This separation ensures that each layer do not directly affect the others layers (baeldung, 2021).

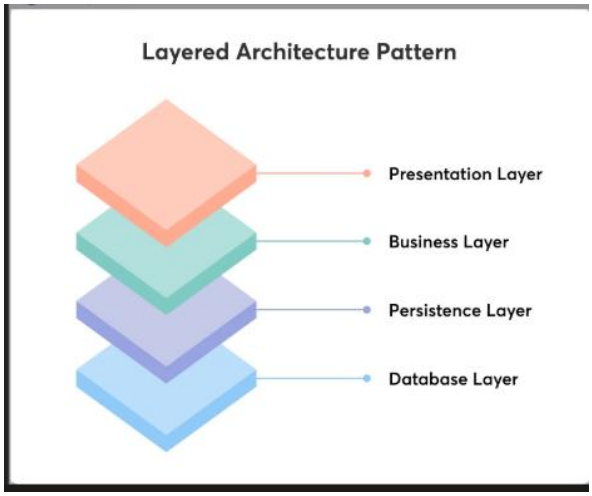


Figure 8:Layered Architectures Pattern

9.2.2 Layers of the system

The Gokyo Bistro Management System will be structured in four layers: -

1. Presentation Layer

This is the top most level that acts as the user interface for the whole system. This is the section that the user will interact with directly and includes the sections such as login page, table booking page, order entry page, menu management page, and feedback page. This section will ensure that information is displayed to the user and sending user inputs to the business layer for processing.

2. Business Layer

The Business layer is the core layer where business rules and application logic are implemented. It analyzes the input from the presentation layer according to the business logic of the system. For instance, in case a user makes a table booking request, the business layer will determine if there is an available table, calculate the advance fee to be paid, and also calculate the amount that should be refunded in case of a booking cancellation.

3. Persistence Layer

This layer controls the interaction between the business layer and the database. This layer handles data storage and retrieval from databases. It manages CRUD operations and interacts with data sources. It serves as a link between the business logic and the database activities.

4. Database Layer

This is the bottom most layer and responsible for storing all the data in the system. It contains all data about users, bookings, orders, menu items, payments, feedback in the Gokyo Bistro Management System.

9.2.3 Justification of Choosing Layered Architecture

The Layered Architecture is selected for this project because it provides a clear separation of concerns and suitable for web-based systems like the Gokyo Bistro Management System, where multiple components interact with each other.

Key reasons include: -

- It is simple and organized which makes easy for the development team to work.
- Each layer has its own specific responsibility which make the system much easier to test and maintain.
- Changes in one layer do not interfere with the other which makes the team can refresh or redesign the user interface without having to modify the business logic or database.

9.3 Design Pattern

A Design pattern are reusable solutions to common problems which occurs quite frequently during the process of software design. Design patterns help developers create more efficient, scalable, and maintainable code. The idea of designs patterns was recognized to the work of 'Gang of Four'- Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. The Gang of Four Designs Patterns are 23 proven solutions to solve software designs problems. And The design pattern chosen for the Gokyo Bistro Management System is the Model-View-Controller (MVC) pattern (AlgoCademy, 2022).

9.3.1 Model-View-Controller (MVC)

Model-View-Controller (MVC) Architecture is one of the most widely used fundamental design pattern in software development separating an application into Model, View and Controller components. This pattern is normally used in software development to create organized and easy-to-maintain code (geeksforgeeks, 2025).

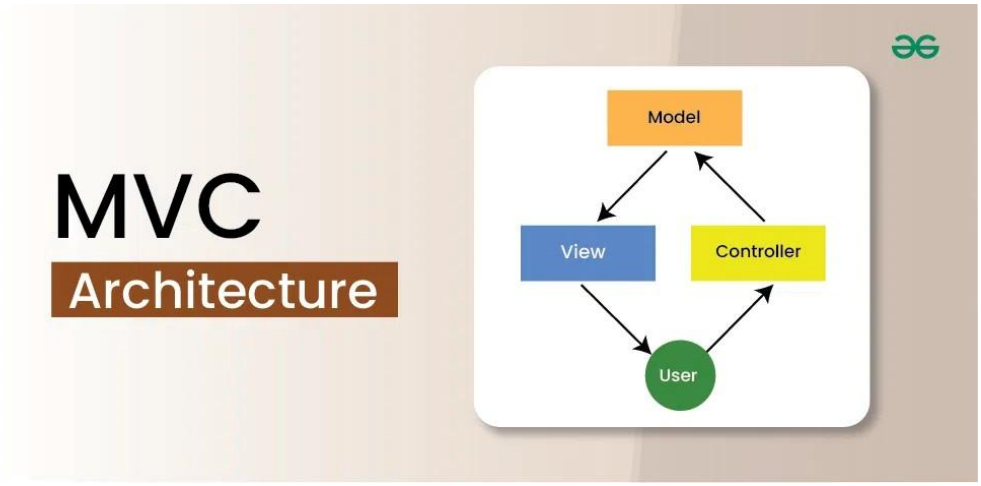


Figure 9:MVC architecure

Components of MVC:

1. Model

The Model is an abstract representation of the data and business logic of the system. It takes care of data management, applying business rule on data and communication with the database. It is unaware about the presentation layer as it does not have any knowledge about how the data would be presented to the user. In Gokyo Bistro Management System, the Models would include:

- User Model: Manages user account data for visitors, members and admin.
- Reservation Model: Manages table booking and cancellation data.
- Order Model: Manages food and beverage order data.
- Payment Model: Manages advance payments and refund calculations.
- Menu Model: Manages menu items including add, update, and delete.
- Feedback Model: Manages ratings and written feedback from users.

2. View

The View is the User Interface Layer of the system. It plays a role of presenting the information given by the Model to the user and transferring the user commands to the Controller. The View is used to display the data to the user in a readable way as the View does not have any business logic it is only meant to display information. For the Gokyo Bistro Management System, the views would include:

- Table Booking and availability page
- Order Placement and home delivery page
- Feedback and rating page
- Login and Registration Page

3. Controller

The controller acts as a bridge between the Model and the View and it takes user input and deciding what to do with it sometimes modify it with the help of the Model and sends back to the view. It is the brains of the application that tie together the Model and View (codecademy, 2022). The controllers used in the Gokyo Bistro Management system are:

- User Controller: Handles login, registration and role-based access.
- Order Controller: Handles order placement, home delivery and online payment.
- Menu Controller: Handles adding, updating and deleting menu items.
- Report Controller: Handles Financial report generation and revenue prediction.
- Reservation Controller: Handles table booking, availability checking and cancellations.
- Feedback Controller: Handles rating submission and feedback storage.

Model View Controller pattern was selected for the design of the Gokyo Bistro Management System since MVC separates the presentation layer, business layer, and the database layer into independent modules. MVC is easy to develop and test in individual parts which is consistent with the Agile Scrum Sprint development methodology that develops software module by module. The View is totally separate from the other two components of the MVC pattern, modifications can be done in the View without making any change in the business logic or database which will help in minimizing the introduction of new bugs. MVC is also widely compatible with web application development tools such as React.js or Java Servlet Page (JSP) for the View and Node.js with Express.js for the Controller and Model respectively which will be utilized to build the current application. Finally, the well-defined architecture provided by MVC makes it much easier to develop and expand the system in the future.

9.4 Development Plan

The development plan outlines the tools, technologies and resources that will be used to build the Gokyo Bistro Management System with priority order of features.

9.4.1 Tools and Technologies

The following tools and technologies have been selected for the development of the system:

- Frontend: React.js and tailwind CSS
- Backend: Node.js and Express.js
- Database: MySQL
- Version Control: GitHub
- Design and Prototyping: Figma
- Testing: Postman

9.4.2 Priority Order of Features

Following the Agile Scrum methodology, the features of the Gokyo Bistro Management System will be developed in the following priority order across seven sprints.

Table 2: Priority Order of Features

Sprint	Features	Priority
Sprint 1	Login & Registration	Highest
Sprint 2	Table Boking and Beverage Selection	High
Sprint 3	Payment and Cancellation Module	High
Sprint 4	Order Placement and Home Delivery	Medium
Sprint 5	Bill Generation and Financial Reports	Medium
Sprint 6	Menu Management and Offers	Medium
Sprint 7	Rating and Feedback System	Lower

9.5 Testing Plan

Testing is the process of evaluating and running the software through the process with the specific objective of identifying any bugs before handling it to the client. The main aim of testing is to identify any bugs and also to ensure the system behaves correctly under all conditions (IBM, 2025). In the Gokyo Bistro Management System, the testing process will take place at every stage of the project development process using the Agile Scrum Framework.

1. Unit Testing
2. Integration Testing
3. Regression Testing
4. Black Box Testing
5. White Box Testing
6. User Acceptance Testing
7. Smoke Testing
8. Performance Testing

9.5.1 Testing Principles

The following principles will guide the testing process throughout the project:

- The main intention of testing is to find errors, not to prove that everything is working perfectly.
- Testing will be carried out by not only by the developers themselves but also by independent testers which approach ensures that the system is reviewed throughout and the results are unbiased.
- The complete testing is impossible so the most important and high-risk test cases will be prioritized.
- A good test case has high probability an error and is nether too simple nor too complex.
- Testing will be integrated into every sprint rather than being left until the end of the project.

9.6 Software Maintenance

Software Maintenance is the process of changing, modifying and updating software that has already been implemented with changes that are aimed to correct errors enhancing performance or making modifications to adapt the system to a changed environment (THALES, 2026). For the Gokyo Bistro Management System, a structured maintenance plan will be followed after deployment to ensure that everything remains smooth and updated as per needs of the organization.

There are four types of Software Maintenance that will be applied for the project:

1. Corrective Maintenance

Corrective Maintenance is the typical classic form of maintenance for software and anything else for that matter that are discovered after the system has been deployed. Even through extensive testing is carried, some issues may only become apparent once real users starts using the system in a live environment. In the case of the Gokyo Bistro Management System, a bug tracking system will be used to log all reported issues. Depending on the seriousness of the problem, each bug will receive a priority rating. Critical bugs related to payment processing and login problems will be corrected instantaneously while minor bugs like display problems will be sorted during regular maintenance.

2. Adaptive Maintenance

Adaptive software maintenance refers to the adjustment or modifying the system in order to ensure compatibility with changes in the external environment such as changes in technology, operating system, cloud storage or other third-party applications as well as policies and rules regarding the software. When these changes are performed, the software must adapt in order to properly meet new requirements and continue to run well. This also includes maintenance necessary as a result of changes in the business requirement for Gokyo Bistro. For instance, Gokyo Bistro wants to add a new payment gateway to its system or process is made easier by the use of Agile Scrum methodology in developing the applications.

3. Perfective Maintenance

Perfective maintenance involves improvements in the system, such that it exceeds the initial requirements increasing its efficiency or usability. Once the system has been operational for some time, the feedback provided by customers and employers will be reviewed to determine if there have been any recommendations for improvement, In cases where there have been suggestions for improvements or where some features have been found hard to navigate, perfective maintenance will be employed to incorporate these changes in the software.

4. Preventive Maintenance

Preventive software maintenance involves making changes to the system in advance in order to prevent any issues that could occur in the future. Code reviews and system audits will be conducted on a regular basis to discover any areas within the codebase that may be problematic over time such as outdated dependencies, inefficient database queries or security vulnerabilities.

10 Protype Development

A prototype is an early visual representation to demonstrate how the user interface would be like even before the commencement o the actual development of the system. This allows the client understand the flow of the system visualize the features and provide feedback at an initial stage. The use of prototyping in software development is an integral part of the process since it avoids developing a system that does not satisfy the needs of the client. In the Gokyo Bistro Management System, a total of 16 prototype screens have been developed covering all the major features outlined in the detailed specification.

10.1 Prototyping tool

Figma, is the software that is being used to create the prototypes. It is an online interface for creating the user interface prototype and design which makes it very useful in creating high-quality mock-ups. It was chosen for this project it is easy to use, supports real time collaboration and produces professional quality designs that closely represent the final system.

10.2 Prototype Screens

1. Login Page

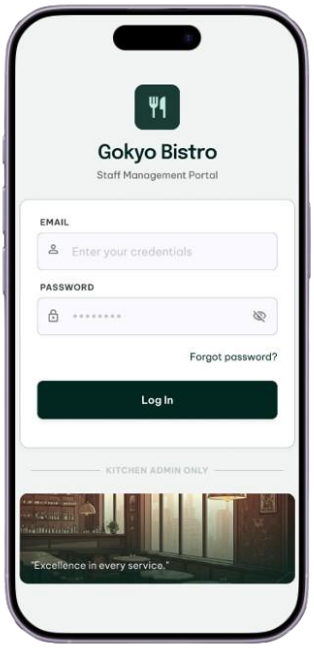


Figure 10: Login page

2. Register Page

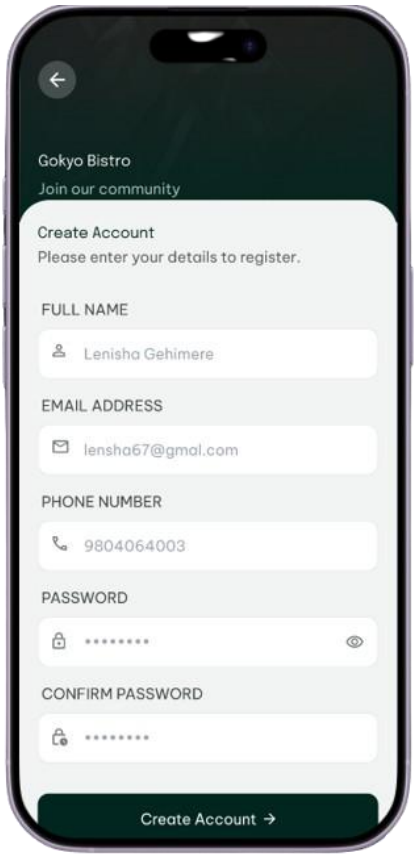


Figure 11:Register Page

3. Home Page

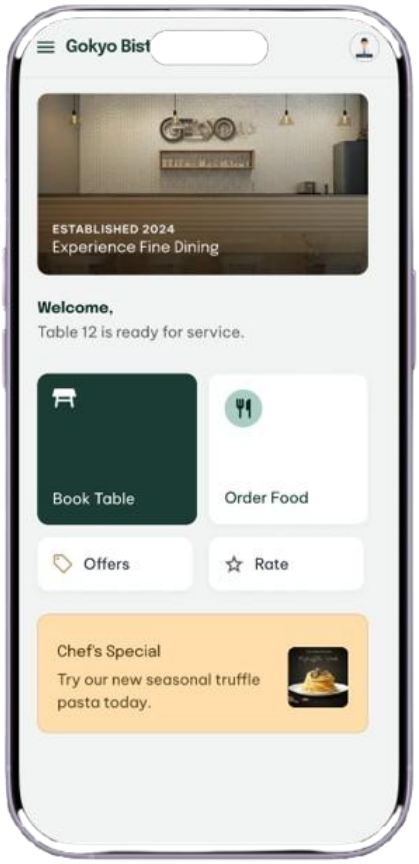


Figure 12:Home Page

4. Check Table Availability

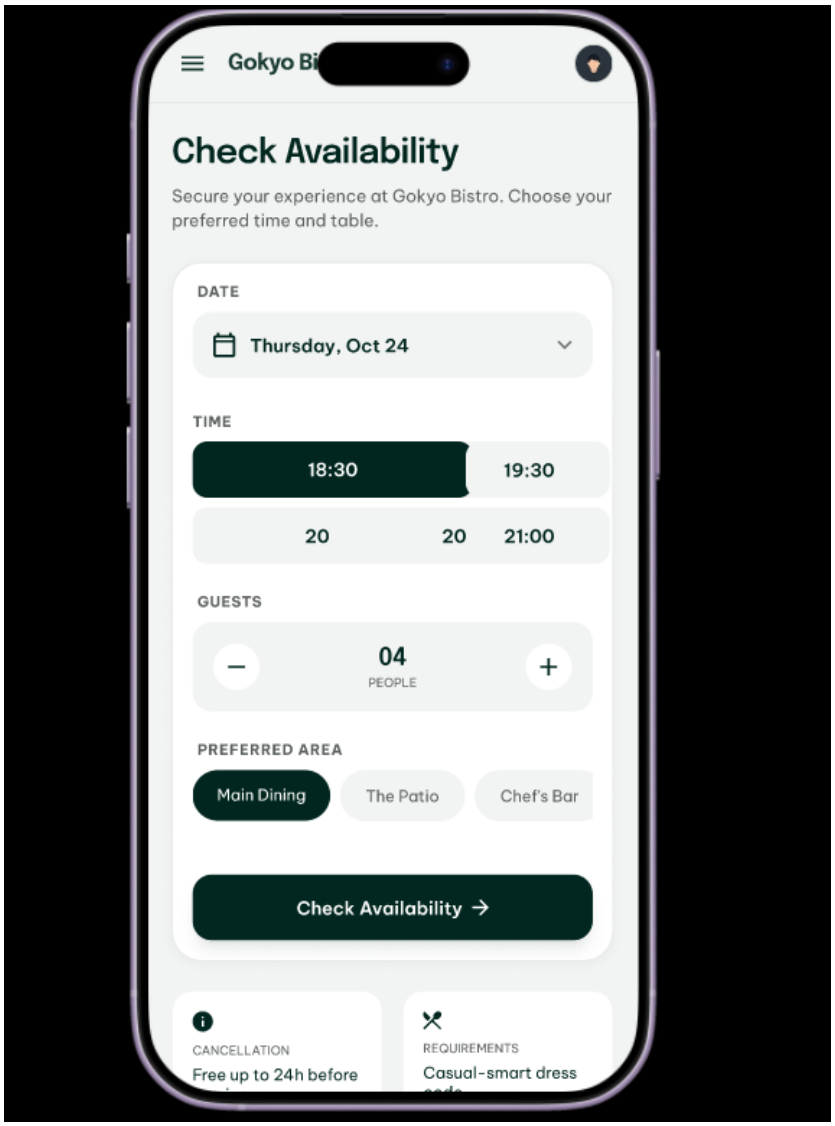


Figure 13:Check table availability

5. Book Table Page

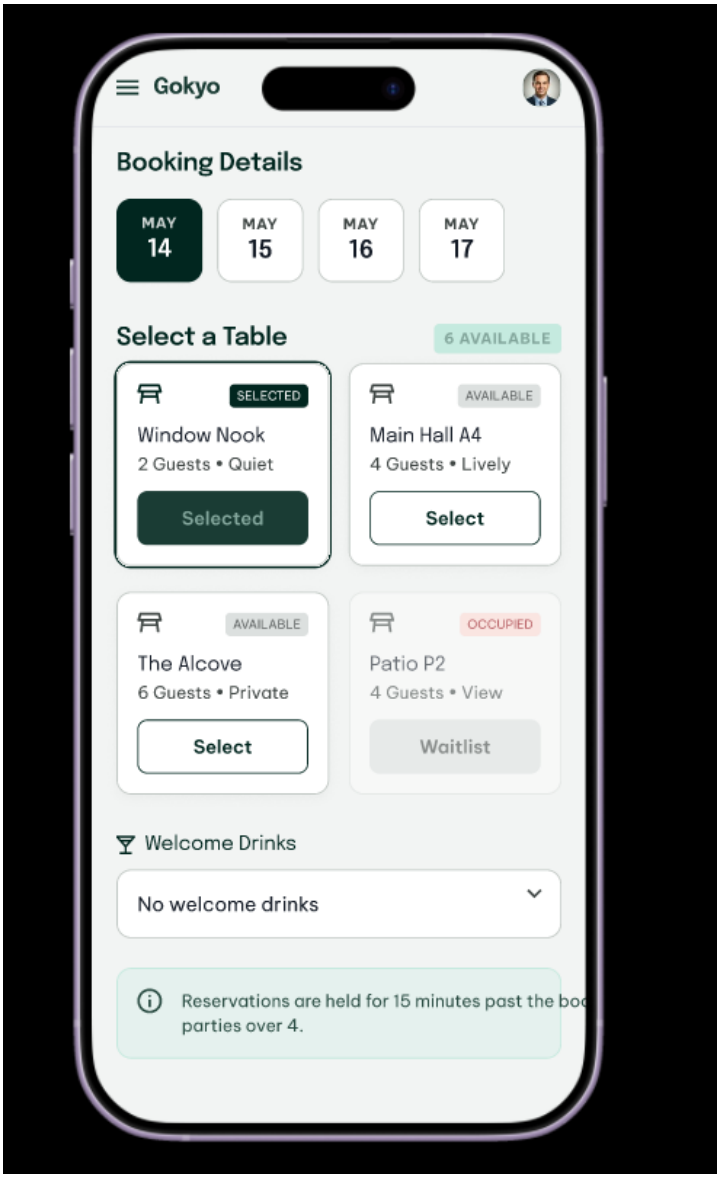


Figure 14:Book table page

6. Payment Page

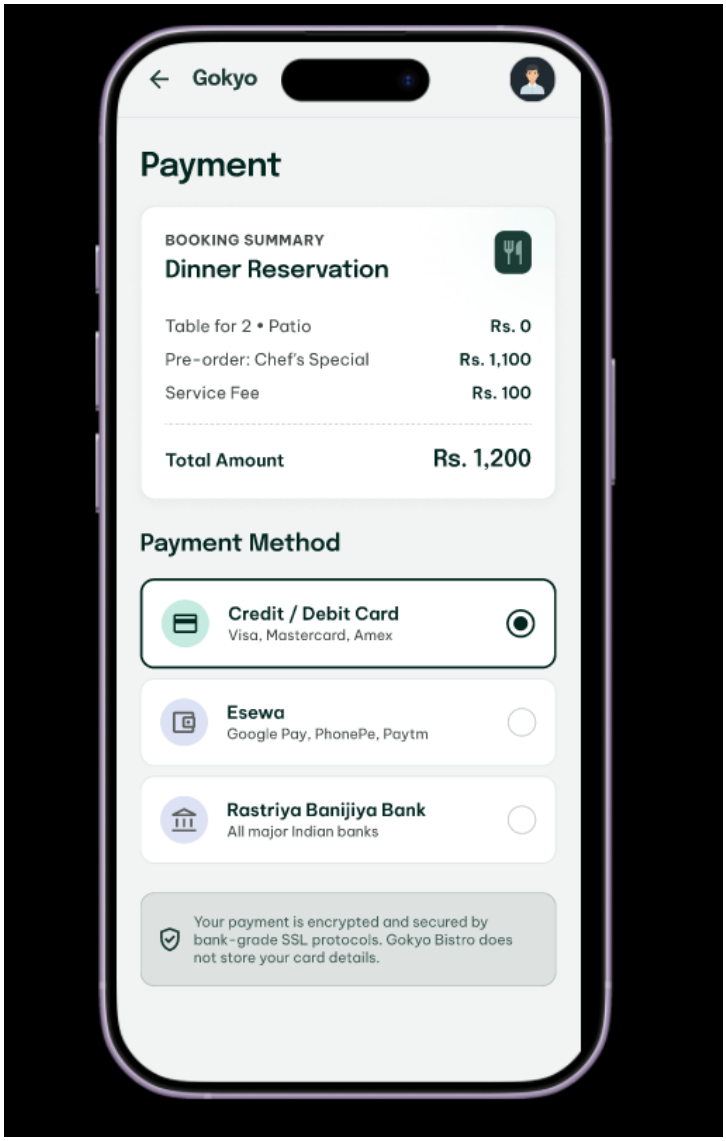


Figure 15:Payment page

7. Booking Confirmation

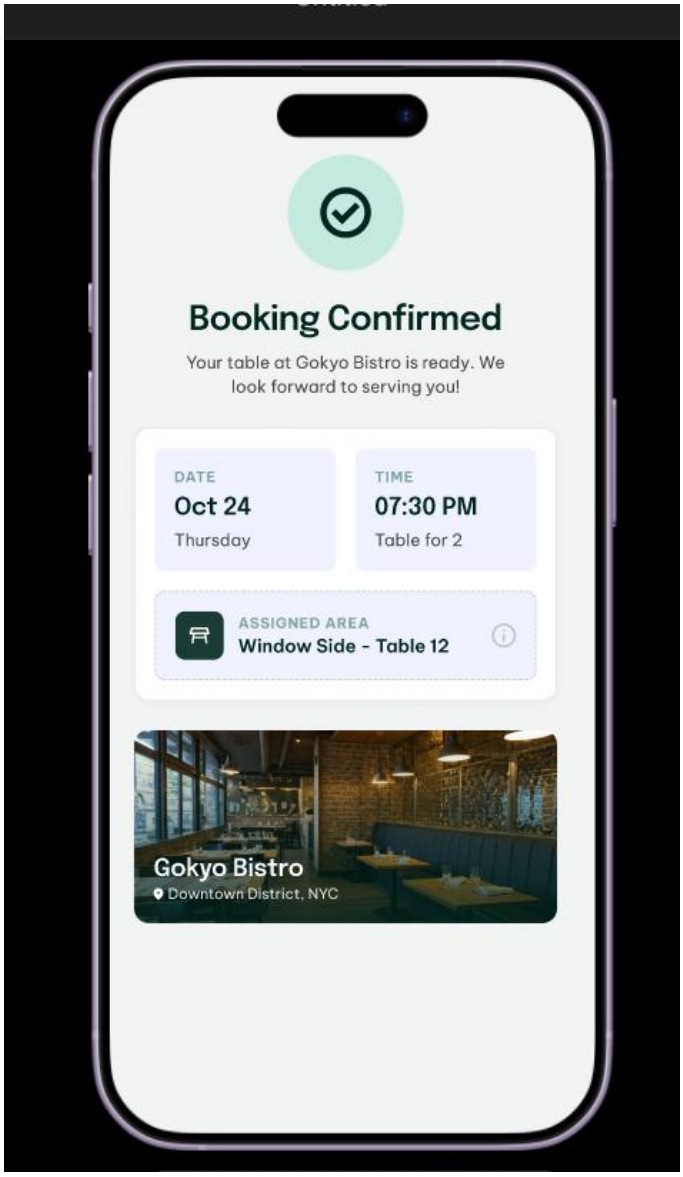


Figure 16:Booking confirmation page

8. Cancel Reservation

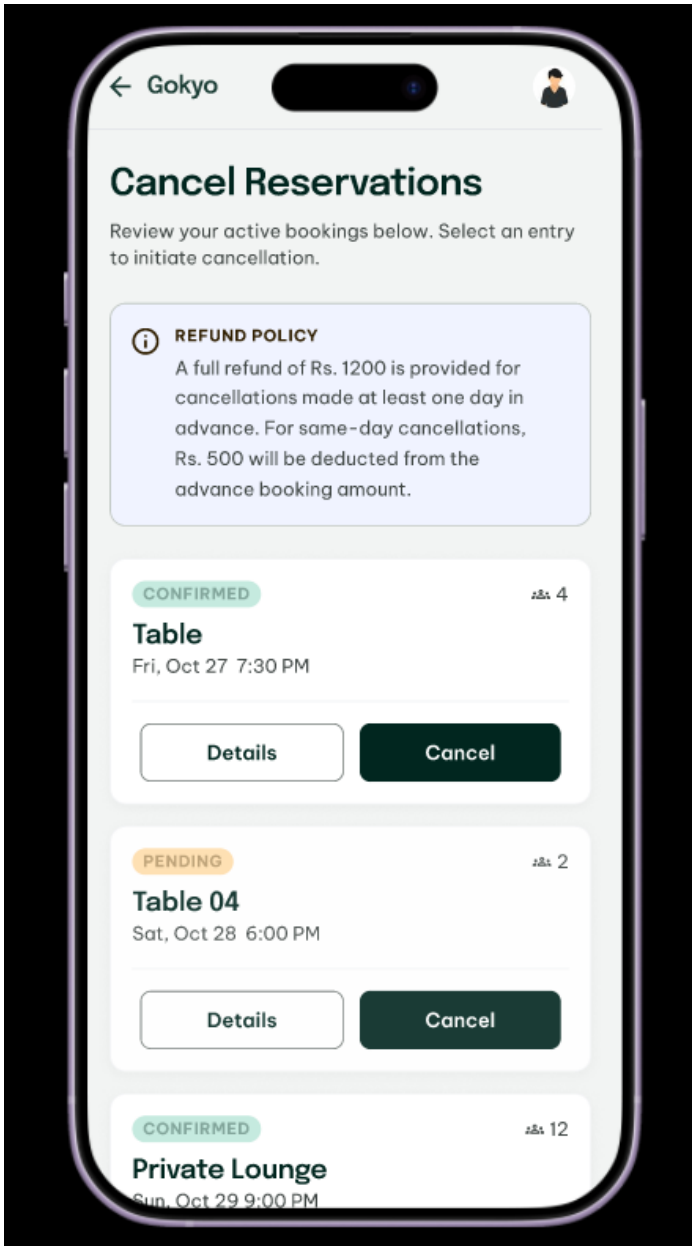


Figure 17:Cancel Reservation Page

9. Place Order Page

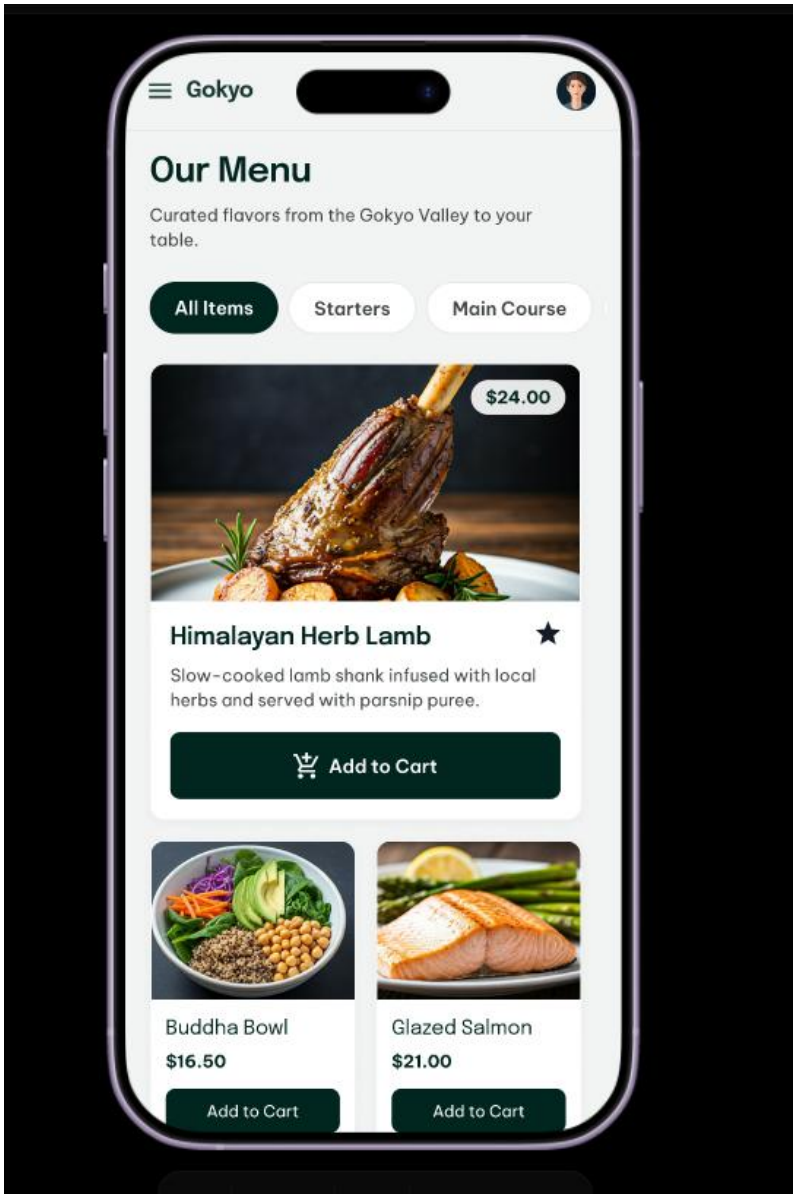


Figure 18:Place Order Page

10. Home Delivery

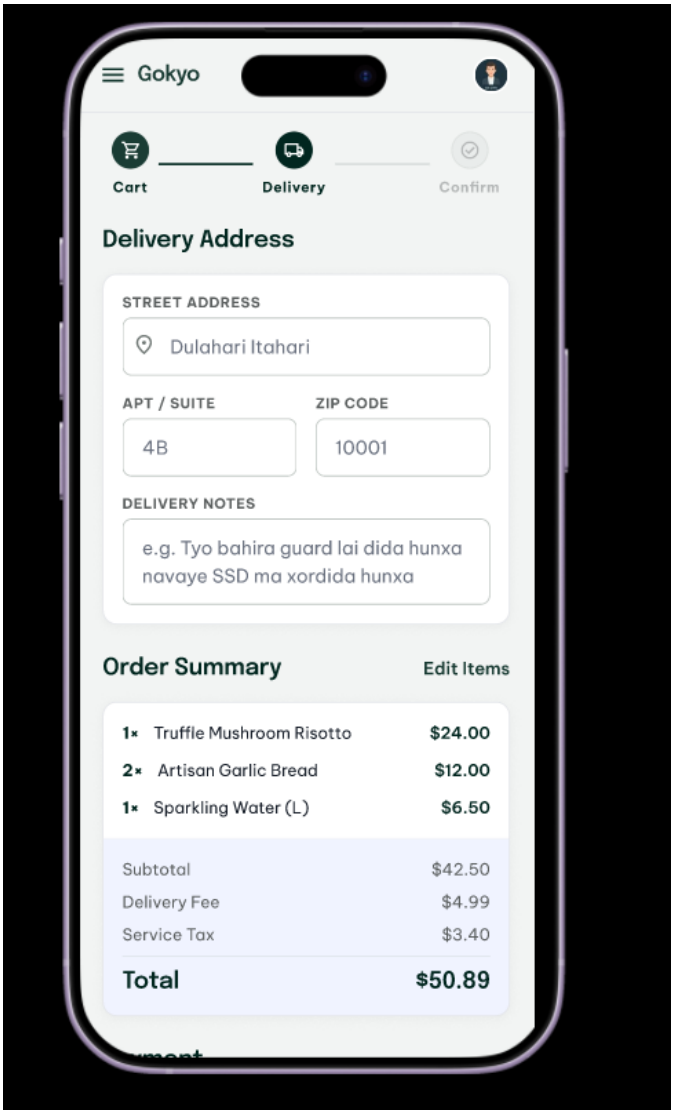


Figure 19:Home Delivery Page

11. Order Confirmation

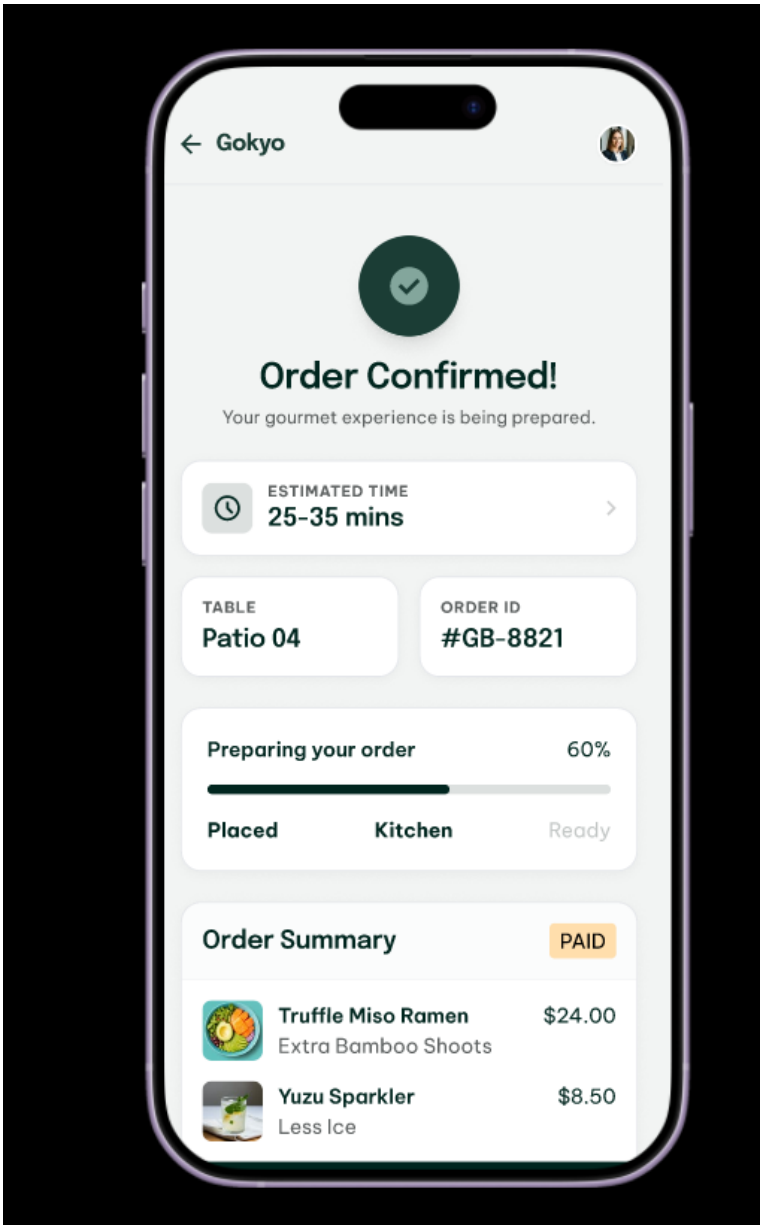


Figure 20:Order Confirmation Page

12. Rate the Experience Page

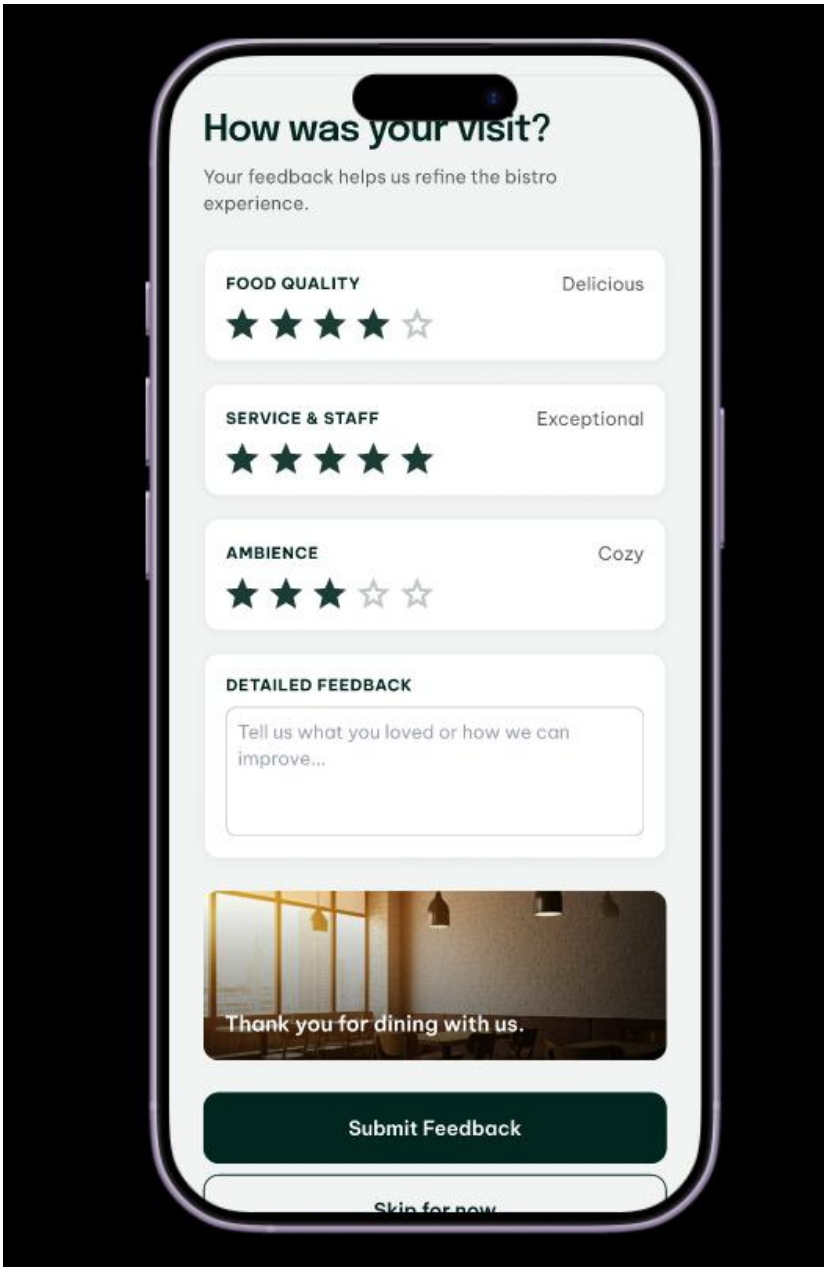


Figure 21:Rate the Experience Page

13. View Offers

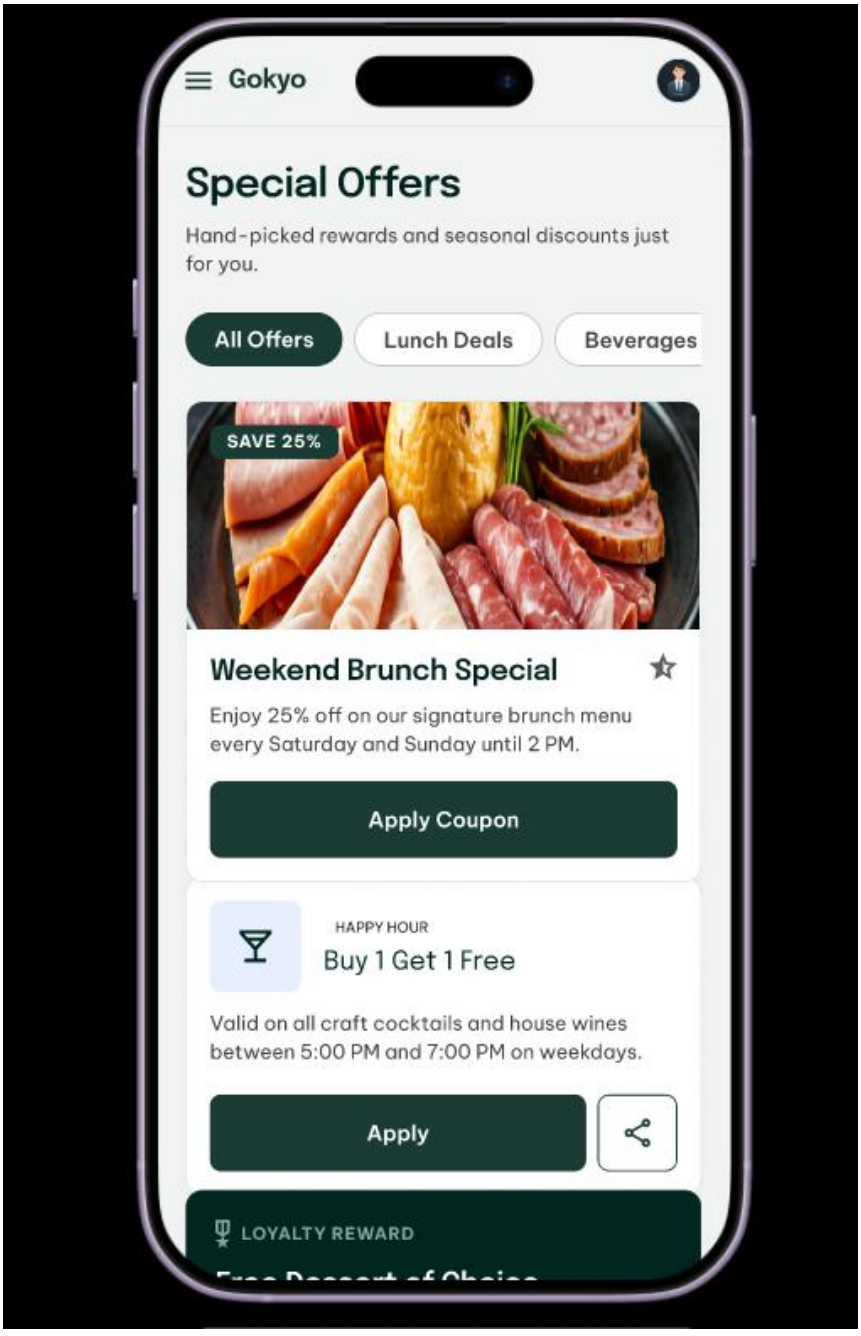


Figure 22:View offers page

14. Admin Dashboard

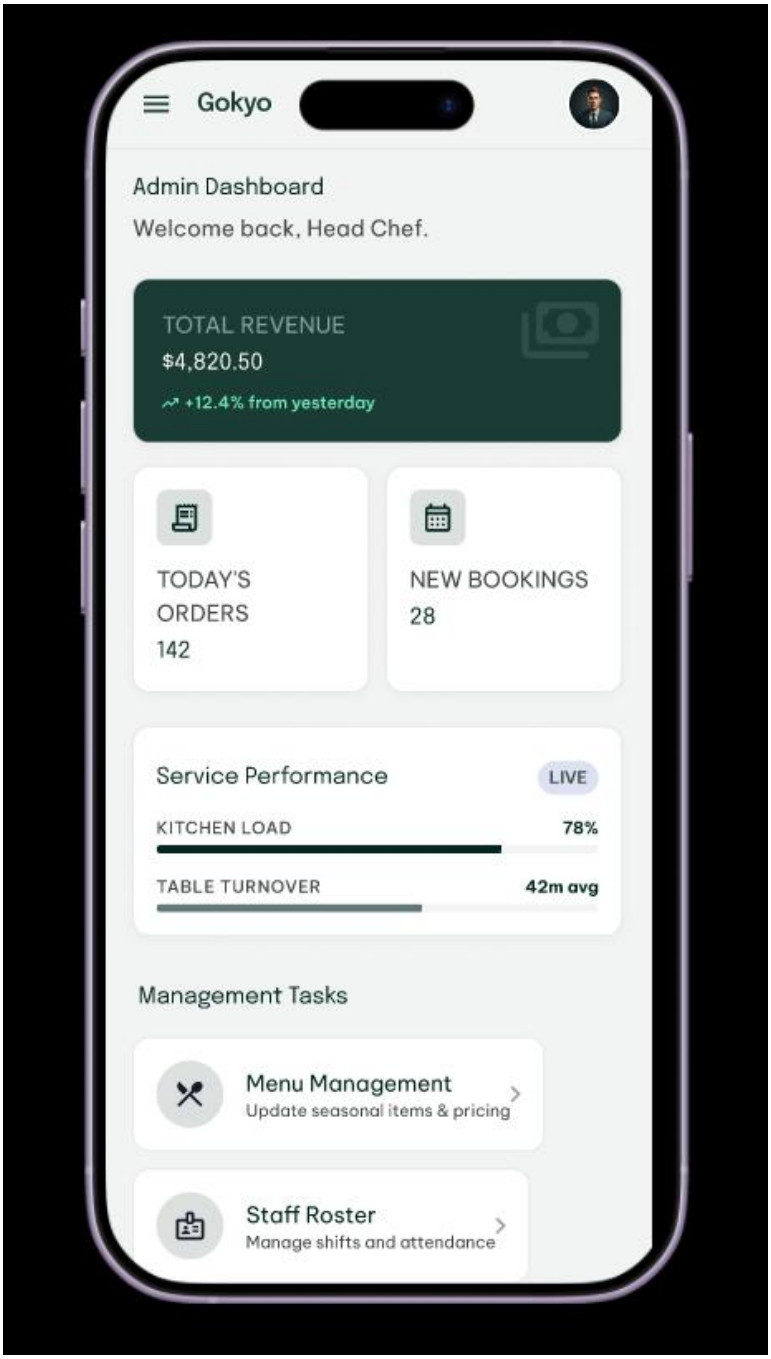


Figure 23:Admin Dashboard Page

15. Menu Management

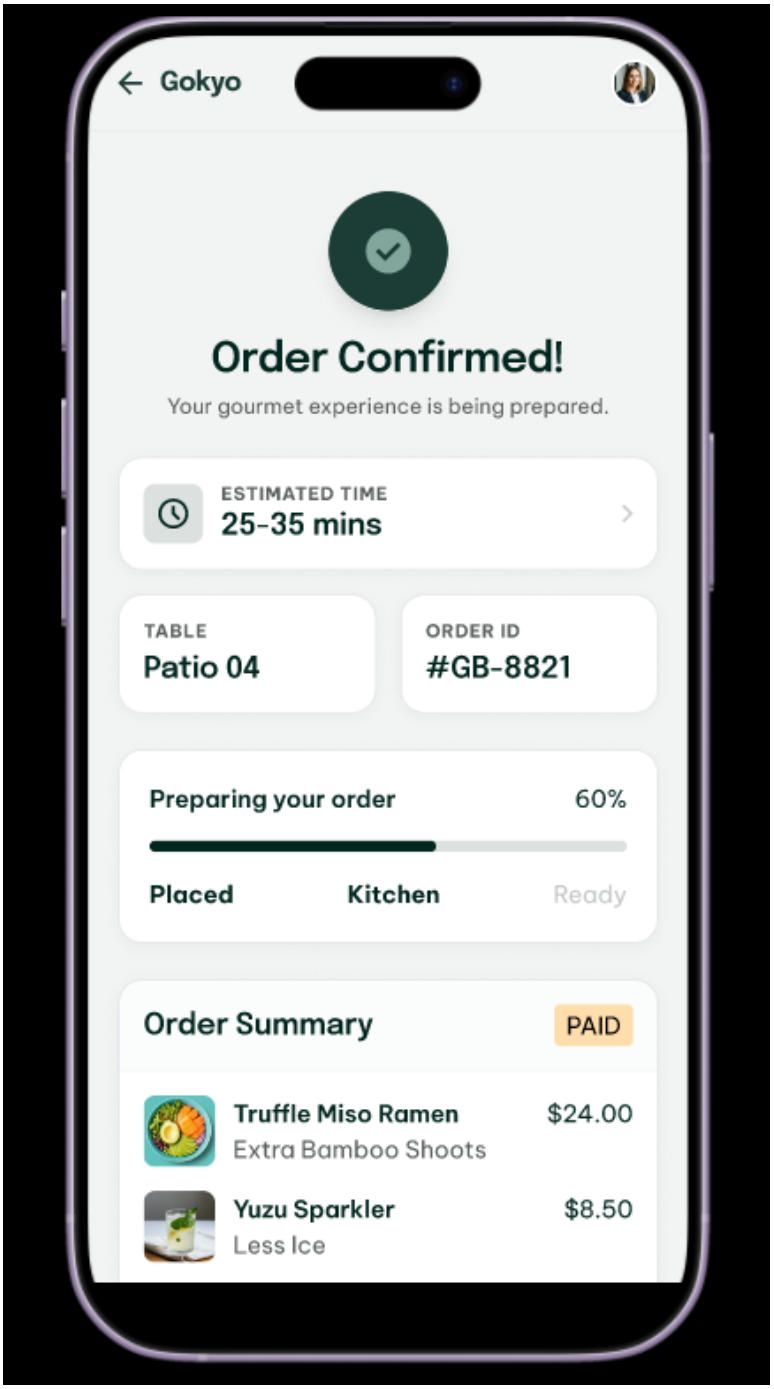


Figure 24:Menu Management Page

16. Report Page

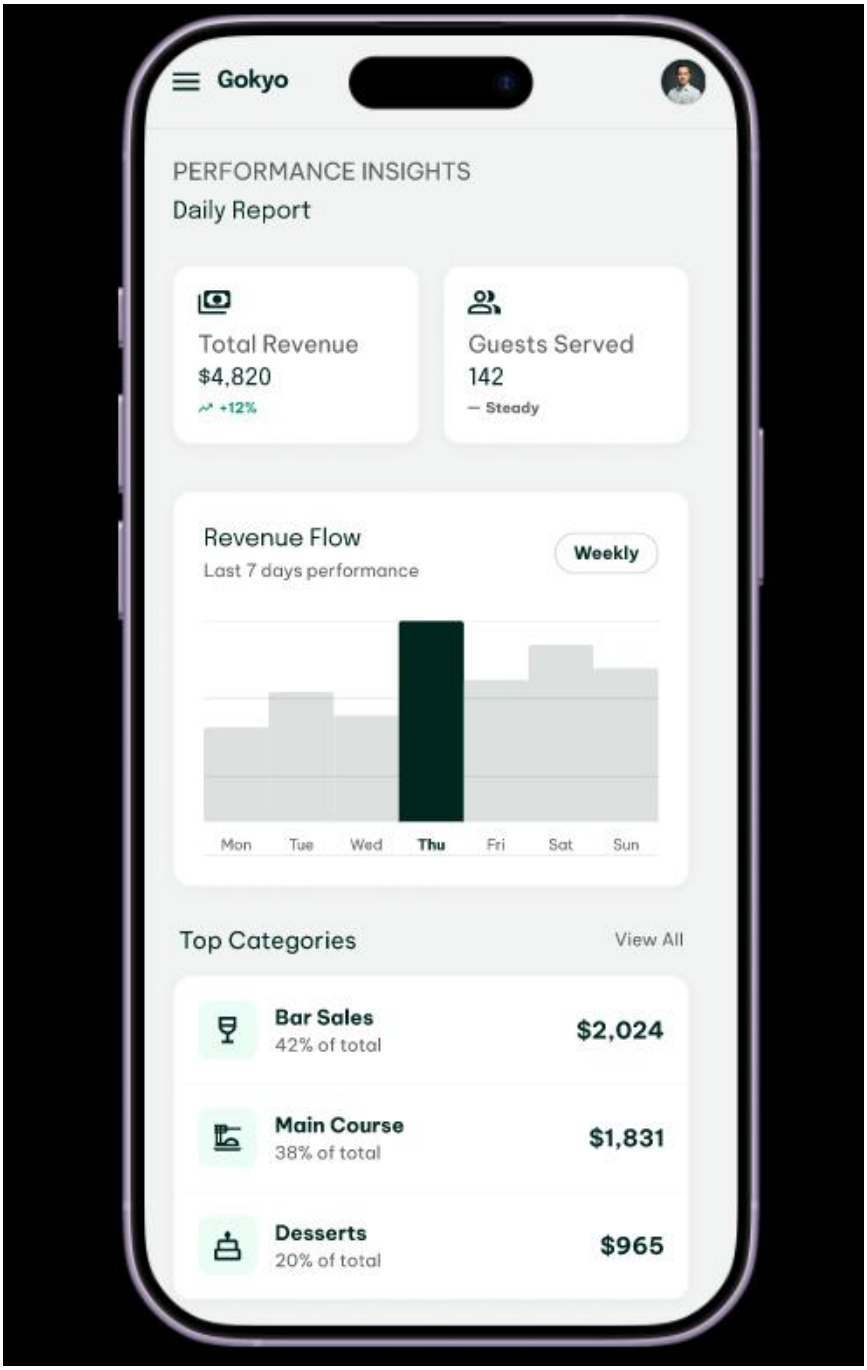


Figure 25:Report page

17. Bill Generation

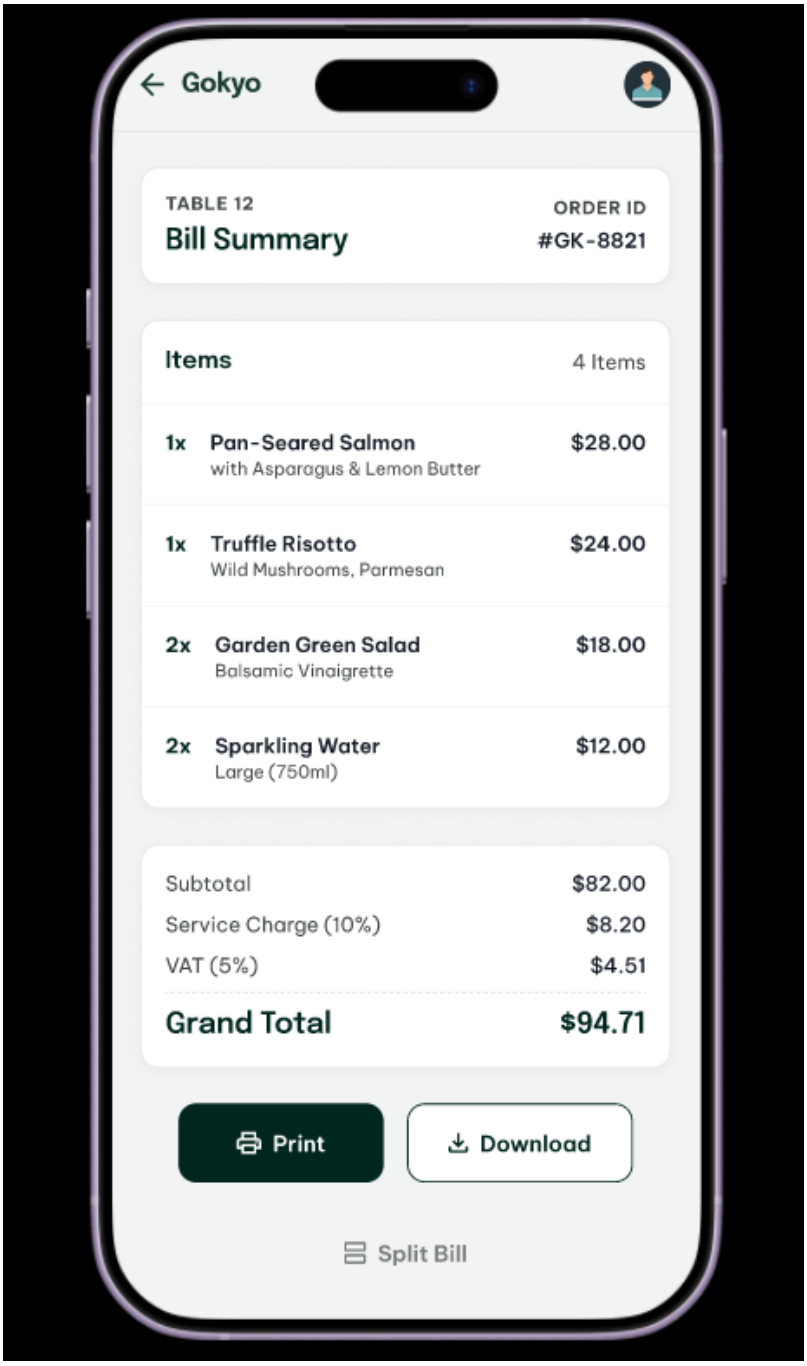


Figure 26:Bill Generation

11 Conclusion

The following report has given an overview of Object-Oriented Analysis and Design for Gokyo Bistro Management System. The purpose of this system is to solve the operational problems that exist at Gokyo Bistro, such as the problem of managing table bookings, placing orders, and making home deliveries because of the increasing customer base.

The report has addressed all the necessary tasks as beginning with the Work Breakdown Structure and Gantt Chart which has placed the project scope and been implemented on the basis of the Agile Scrum approach. A Use Case Model has been created that showed all of the actors and use cases of the system, a Sequence Diagram of the Cancel Reservation use case, and an Activity Diagram of the Rate the Experience use case. A Class Diagram with 15 domain classes was generated which illustrates all the relationships between the system entities with inheritance, aggregation and composition.

The Further Development Plan defined the architectural decision, design pattern, development plan, testing plan and maintenance plan to inform the reality implementation of the system. Last but not least, 17 prototype screens were created in order to visually illustrate the key features of the system.

Overall, this report has successfully addressed all the requirements o the coursework and has provided a solid analysis and design foundation for the development of the Gokyo Bistro Management System.

12 References

AlgoCademy. (2022) *understanding-design-patterns-and-when-to-use-them* [Online]. Available from: https://algocademy.com/blog/understanding-design-patterns-and-when-to-use-them/ [Accessed 28 April 2026].

baeldung. (2021) *layered-architecture* [Online]. Available from: https://www.baeldung.com/cs/layered-architecture [Accessed 28 April 2026].

codecademy. (2022) *mvc-architecture-model-view-controller* [Online]. Available from: https://www.codecademy.com/article/mvc-architecture-model-view-controller [Accessed 28 April 2026].

geeksforgeek. (2026) *unified-modeling-language-uml-activity-diagrams* [Online]. Available from: https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-activity-diagrams/ [Accessed 28 April 2016].

geeksforgeeks. (2024) *software-engineering-work-breakdown-structure* [Online]. Available from: https://www.geeksforgeeks.org/software-engineering/software-engineering-work-breakdown-structure/ [Accessed 21 April 2026].

geeksforgeeks. (2025) *mvc-architecture-system-design* [Online]. Available from: https://www.geeksforgeeks.org/system-design/mvc-architecture-system-design/ [Accessed 28 April 2026].

geeksforgeeks. (2026) *scrum-software-development* [Online]. Available from: https://www.geeksforgeeks.org/software-engineering/scrum-software-development/ [Accessed 28 April 2026].

geeksofgeek. (2025) *fundamentals-of-software-architecture* [Online]. Available from: https://www.geeksforgeeks.org/software-engineering/fundamentals-of-software-architecture/ [Accessed 28 April 2026].

geeksOfgeeks. (2026) *use-case-diagram* [Online]. Available from: https://www.geeksforgeeks.org/system-design/use-case-diagram/ [Accessed 18 April 2026].

geeksOfgeeks. (2026) *unified-modeling-language-uml-sequence-diagrams* [Online]. Available from: https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-sequence-diagrams/ [Accessed 24 March 2026].

IBM. (2025) *software-testing* [Online]. Available from: https://www.ibm.com/think/topics/software-testing [Accessed 29 April 2026].

THALES. (2026) *four-types-of-software-maintenance* [Online]. Available from: https://cpl.thalesgroup.com/software-monetization/four-types-of-software-maintenance [Accessed 29 April 2026].